Learning Eigenstructures of Unstructured Data Manifolds

Roy Velich[1*] Arkadi Piven[1] David Bensaïd[1] Daniel Cremers[2,3] Thomas Dagès[1,2,3] Ron Kimmel[1]

In memory of Haïm Brezis (1944 – 2024).

Abstract

We introduce a novel framework that directly learns a spectral basis for shape and manifold analysis from unstructured data, eliminating the need for traditional operator selection, discretization, and eigensolvers. Grounded in optimal-approximation theory, we train a network to decompose an implicit approximation operator by minimizing the reconstruction error in the learned basis over a chosen distribution of probe functions. For suitable distributions, they can be seen as an approximation of the Laplacian operator and its eigendecomposition, which are fundamental in geometry processing. Furthermore, our method recovers in a unified manner not only the spectral basis, but also the implicit metric's sampling density and the eigenvalues of the underlying operator. Notably, our unsupervised method makes no assumption on the data manifold, such as meshing or manifold dimensionality, allowing it to scale to arbitrary datasets of any dimension. On point clouds lying on surfaces in 3D and high-dimensional image manifolds, our approach yields meaningful spectral bases, that can resemble those of the Laplacian, without explicit construction of an operator. By replacing the traditional operator selection, construction, and eigendecomposition with a learning-based approach, our framework offers a principled, data-driven alternative to conventional pipelines. This opens new possibilities in geometry processing for unstructured data, particularly in high-dimensional spaces.

1. Introduction

Differential geometry is at the core of shape and manifold analysis. By defining operators, one can encode the underlying geometry and compute intrinsic and extrinsic quantities, from geodesics and Gaussian curvature to mean curvature and normals. The specific choice of operator depends on the task at hand. For example, the Laplace-Beltrami operator (LBO) captures intrinsic geometric properties of the underlying manifold and governs heat diffusion. Hamiltonian operators, appearing in the famous Schrödinger equation, model wave motion and allow potential-modulated particle dynamics. The biharmonic operator encodes fourth-order behavior and is natural for smooth interpolation and shape-aware distances.

In practice, these operators are primarily used for their spectral decompositions, which serve as the computational foundation for geometry processing. For instance, LBO eigenvalues are intrinsic shape signatures [50, 51, 62, 102, 107], while LBO eigenfunctions power heat- and wave-kernel methods for smoothing and propagation [35, 55], define feature descriptors [7, 113, 123], and support matching via functional maps [93]. As such, these eigendecompositions are routinely required in geometry processing.

The standard pipeline for computing operator eigendecompositions on discrete data begins by choosing a discrete approximation that preserves key geometric properties such as positive semi-definiteness (PSD), symmetry, consistency, or weak formulations [100, 118, 135]. Then, explicitly construct a matrix acting as the discretized linear operator and solve a generalized eigenvalue problem using a classical numerical solver. Although effective on data sampled from surfaces in 3D, this pipeline designed for intrinsically two-dimensional manifolds does not scale gracefully to high-dimensional manifolds [118].

Here, we propose a different route. Rather than targeting a specific operator, explicitly discretizing and eigendecomposing it, we learn the spectral decomposition directly from the data. The learned spectral decomposition implicitly corresponds to an operator that is reconstructible a posteriori from the basis and associated eigenvalues. Building on optimal-approximation theory [3, 4, 24], we find the eigenbasis of an optimal-approximation operator, that minimizes the reconstruction error over a class of probe functions. Different probe classes induce different metrics and thus different operators. In particular cases, the learned operator approximates the LBO, but the proposed framework generalizes to more operators. While only the eigenbasis is learned, the eigenvalues are given directly as a by-product of the worst-case reconstruction error. This yields a learning methodology for spectral analysis that is data-driven and

*Corresponding author: royve@campus.technion.ac.il
[1]Technion - Israel Institute of Technology
[2]Technical University of Munich
[3]Munich Center for Machine Learning

avoids the need for explicit operator construction.

To summarize, our contributions are as follows:

- We learn spectral bases directly from point clouds, bypassing the explicit choice of an operator, its construction, and classical numerical eigensolvers. We ground the method in an optimal-basis representation theory and use it to propose a unique learning objective.
- We demonstrate on surfaces in 3D that the proposed approach provides eigenstructures and estimated metrics performing competitively with oracle discrete LBO baselines, while bypassing explicit operator construction.
- We show that the method scales from surfaces to higher-dimensional data manifolds, enabling scalable manifold learning where mesh-based pipelines are inapplicable and common graph-based ones are unreliable.
- We provide our code at https://github.com/royvelich/learning-eigenstractures.

2. Related Works

Our framework lies at the intersection of operator discretization and neural methods for eigenproblems.

Discrete operators. In geometry processing, discrete operators supply via spectral decomposition [28] the orthonormal basis that we actually use to process signals on meshes and graphs. For instance, filtering [15, 71], diffusion [35, 119], and convolution [27, 72] are usually implemented in this spectral basis. Notably, the community rarely applies the operator matrix directly. In practice, only the first eigenvectors are kept, which amounts to projecting a signal onto the lowest-energy subspace defined by the operator. This acts as a low-pass filter where only the (smooth) low-frequency components are retained. Because the operator defines what "energy", "smoothness", and "frequency" mean, different operators emphasize different properties of the manifold. This motivates the importance of choosing operators wisely in the field of geometry processing.

Perhaps the most important example, the LBO [12, 109] is ubiquitous in geometry processing [20–22, 26, 30, 123, 139, 145]. Discretizing the weak form gives the cotangent Laplacian [87, 100] – the reference in LBO discretization, but alternatives exist [32, 142]. Cotangent weights need well-triangulated meshes, i.e. Delaunay, to avoid negative edges violating the maximum principle [135]. They cannot be directly applied to unstructured data, like point clouds, requiring first wise meshings, e.g. with the tufted cover of the Robust Laplacian [118] enabling flips to Delaunay triangulations. While it extends to thin 3D volumes, this method does not scale beyond surfaces. For higher-dimensional manifolds, graph Laplacians [33, 65] are used, but they depend strongly on connectivity, e.g. local density, unlike the targeted smooth LBO [73]. Constructing a reliable high-dimensional generalization of the LBO is a challenge.

Learning Laplacians has become a popular line of research as the amount of data increases. Some works suggest learning the operator action [103] but lack eigendecompositions. Others learn eigenvalues but not eigenvectors [5]. Most learn a Laplacian matrix from data rather than heuristics [95, 141], but they still explicitly approximate the operator by assembling mass and stiffness matrices and then apply eigensolvers sensitive to these approximations. As they rely on triangle-based discretizations, they target surfaces and do not extend to higher-dimensionality. For implicit neural representations, the Laplacian can be built via Rayleigh quotients directly [140]. However, this assumes a two-dimensional surface and the Euclidean ambient metric, removing adaptivity to other metrics or dimensionalities. On the theoretical front, Laplacian estimation has progressed [29, 88, 98, 128], yet, translating these insights into practice remains largely unexplored.

Additionally, other discrete operators provide different insights. Changing the metric, via scaling (scale-invariant LBO [2, 20, 54, 117]), anisotropy [6, 17, 18, 106, 109], or asymmetry [9, 37–39, 91, 136], changes the LBO and its approximation. Beyond LBOs, both intrinsic [14, 31, 105] and extrinsic [134] alternatives are understudied.

Recent works have also focused on making spectral methods more robust to the arbitrary choice of eigenbasis for a given operator, either by defining spectral processing on eigenspaces [80] or by adaptive canonicalization [77]. However, these methods still begin from a predefined operator and its spectral decomposition.

Eigenproblems and neural networks. Recent efforts apply neural networks to operator eigenvalue problems, typically using variational or dynamical formulations. In [111], networks are trained to represent individual Laplacian eigenfunctions as continuous functions on parametrized domains, based on the Rayleigh quotient objective and finding eigenfunctions sequentially via Gram-Schmidt orthogonalization, an idea that can be extended to learning multiple eigenfunctions simultaneously [13]. However, this approach is limited to simplistic impractical domains, e.g. Euclidean. Another approach reformulates the eigenvalue problem as a fixed point of the operator's semi-group flow, training networks via forward-backward stochastic differential equations to handle high-dimensional problems, up to ten dimensions [56]. These methods fundamentally differ from the proposed approach: (1) they require explicit domain parameterization with global coordinates, (2) they assume flat geometry by taking Euclidean gradients via automatic differentiation, and (3) they must be trained from scratch for each specific domain, with no mechanism for generalization across different geometries. Thus, they cannot directly handle curved manifolds sampled as point clouds without manually specifying metric tensors

and managing coordinate singularities.

3. Method

3.1. Foundations

Denote $\mathcal{M} \subset \mathbb{R}^d$ as a high-dimensional manifold, typically a curved low-dimensional surface embedded in $\mathbb{R}^d$, with functions $f \in \mathcal{F}(\mathcal{M}, \mathbb{R})$ and linear operators $L : \mathcal{F}(\mathcal{M}, \mathbb{R}) \to \mathcal{F}(\mathcal{M}, \mathbb{R})$ acting on them. When the manifold is discretely sampled by n points, scalar functions can be represented by vectors $f \in \mathbb{R}^n$ and operators as matrices $L \in \mathbb{R}^{n \times n}$. Here, we denote by $\langle \cdot, \cdot \rangle_L$ the L-weighted inner product $\langle f, g \rangle_L = f^\top L g$ for any *symmetric positive definite* (SPD) matrix L, with its induced norm $\|f\|_L = \sqrt{\langle f, f \rangle_L} = \sqrt{f^\top L f}$.

Optimal-approximation theory. Given a class $\mathcal{C}$ of signals on the discrete domain $\mathcal{M}$, a fundamental question arises: What is the optimal orthonormal basis for representing functions $f \in \mathcal{C}$? The answer depends on how we characterize the class of signals. A key insight [4, 24] is that many practical signal classes can be characterized by constraints of the form $\|f\|_L \leq 1$, where L is an SPD operator encoding prior knowledge about the signals. Remarkably, for any such constraint class, the optimal basis is given by the eigenvectors of L itself.

Theorem 3.1 (Min-Max Optimality [4, Theorem 2.1])**.** *Given a symmetric positive definite operator L with eigenvalues $0 < \lambda_1 \leq \cdots \leq \lambda_n$ and eigenvectors $e_1, \ldots, e_n$, the min-max approximation error*

$$\alpha_k \quad = \quad \min_{b=(b_1,\ldots,b_n)} \max_{\|f\|_L \leq 1} \left\| f - \sum_{i=1}^{k} \langle f, b_i \rangle b_i \right\|^2 ,$$

where b ranges over orthonormal bases, is minimized by the first k eigenvectors of L, i.e. $b_i = e_i \ \forall i \leq k$, with optimal value $\lambda_{k+1} = \frac{1}{\alpha_k}$. For simple spectrum, $\lambda_i < \lambda_{i+1} \ \forall i$, the basis b that is a solution for every k is unique up to signs.

The key insight is that for any SPD operator $L \in \mathbb{R}^{n \times n}$, the optimal orthonormal basis for the progressive k-term approximation, i.e. for every k, is uniquely given by the eigenvectors of L ordered by increasing eigenvalues. Crucially, the worst approximation error using the first k eigenvectors is $\frac{1}{\lambda_{k+1}}$, meaning that the $(k+1)$-th eigenvalue is inversely proportional to the worst maximum reconstruction error.

The min-max formulation (Theorem 3.1) optimizes over the set $\mathcal{C}_L = \{f \,;\, \|f\|_L \leq 1\}$. Instead of minimizing the worst-case reconstruction error, another natural approach to find optimal representations would be to maximize the captured variance. This alternative problem leads to Principal Component Analysis (PCA) on the same class of signals $\mathcal{C}_L$. Remarkably, we obtain the following result when performing PCA on uniformly distributed signals from $\mathcal{C}_L$.

Theorem 3.2 (Operator-Bounded PCA [4, Section 5])**.** *Given a symmetric positive definite operator L with eigenvalues $0 < \lambda_1 \leq \cdots \leq \lambda_n$, and eigenvectors $e_1, \ldots, e_n$, the PCA objective*

$$\min_{b=(b_1,\cdots,b_n)} \mathbb{E}_{f \sim \mathcal{U}\left(\|f\|_L \leq 1\right)} \left(\left\| f - \sum_{i=1}^{k} \langle f, b_i \rangle b_i \right\|^2 \right) ,$$

over orthonormal bases b is minimized by the eigenvectors of the covariance matrix $R_L = \mathbb{E}_{f \sim \mathcal{U}\left(\{\|f\|_L \leq 1\}\right)}[f f^\top]$, which are the first k eigenvectors of L, with eigenvalues (variances) $\lambda_1^{-1} \geq \cdots \geq \lambda_n^{-1}$. In other words: $R_L = L^{-1}$.

For a reminder why the expectation of the approximation error is PCA, i.e. iterative maximisation of the Rayleigh quotient of R_L, see Appendix A.1. Therefore, PCA on $\mathcal{C}_L$ yields the same eigenvectors and order as the min-max solution. The min-max optimization (Theorem 3.1) and its PCA counterpart (Theorem 3.2) are thus equivalent when applied to the same signal class. These dual formulations have useful practical implications that we exploit in our method.

A natural prior for signals f on a manifold is smoothness, implying that the Dirichlet energy $\|\nabla f\|_2$ is bounded. By Green's identity, $\|\nabla f\|_2 = \|f\|_\Delta$, where Δ is the (discrete) Laplace-Beltrami operator (LBO). Thus, bounding the Dirichlet energy leads to choosing Δ for L. To motivate our method, let us thus focus on the Laplacian operator and its construction. This will enable the derivation of our framework, generalizable both beyond Laplacian operators and to arbitrarily high-dimensional manifolds, from surfaces in $\mathbb{R}^3$ to image datasets.

Discrete Laplacians. The discrete Laplacian is traditionally constructed from two fundamental matrices, the mass matrix $M \in \mathbb{R}^{n \times n}$, which is a positive diagonal matrix encoding the local (metric-dependent) sampling density defining the manifold's Riemannian metric, and the stiffness matrix $S \in \mathbb{R}^{n \times n}$ encoding geometric relationships that discretize the continuous Laplace-Beltrami operator. For two-dimensional surfaces, M represents vertex areas and S contains cotangent weights [6, 19, 121]. For higher-dimensionality, the LBO is harder to approximate, thus M and S are constructed with schemes more sensitive to the connectivity of the relationship graph. In any case, the matrices M and S enable two common formulations for the discretized Laplacian: the unnormalized Laplacian $L = M^{-1}S$ and the symmetric normalized Laplacian $L_{\text{norm}} = M^{-\frac{1}{2}} S M^{-\frac{1}{2}}$. While both operators share identical eigenvalues, their eigenvectors differ. The unnormalized Laplacian's eigenvectors $\mathbf{v}_i \in \mathbb{R}^n$ are M-orthogonal, satisfying $\langle \mathbf{v}_i, \mathbf{v}_j \rangle_M = \delta_{ij}$, where $\delta_{ij} = 1$ if $i = j$ and 0 otherwise, whereas the normalized Laplacian's eigenvectors $\mathbf{q}_i \in \mathbb{R}^n$ are Euclidean-orthogonal, with $\langle \mathbf{q}_i, \mathbf{q}_j \rangle = \delta_{ij}$.

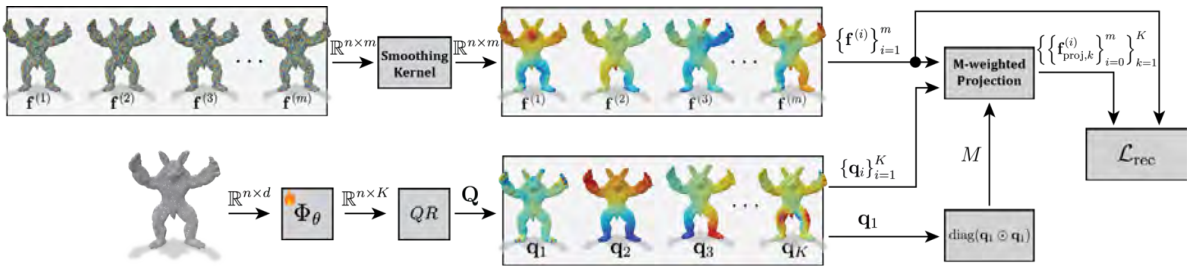


Figure 1. Overview of our neural framework to compute spectral bases directly from unstructured point clouds of any dimensionality, based on optimal-approximation theory, without first explicitly choosing, discretely approximating, and eigendecomposing an operator.

Both sets of eigenvectors are related by $\mathbf{v}_i = M^{-\frac{1}{2}}\mathbf{q}_i$, as $L(M^{-\frac{1}{2}}\mathbf{q}_i) = M^{-\frac{1}{2}}M^{-\frac{1}{2}}SM^{-\frac{1}{2}}\mathbf{q}_i = \lambda_i M^{-\frac{1}{2}}\mathbf{q}_i$.

On closed connected manifolds[1], these operators' nullspaces reveal important differences. While the vector of ones $\mathbf{1}$ lies in the nullspace of L, the vector $M^{\frac{1}{2}}\mathbf{1}$ lies in the nullspace of L_{norm}. This means the first eigenvector $\mathbf{q}_1$ of L_{norm} corresponding to eigenvalue zero is proportional to $M^{\frac{1}{2}}\mathbf{1}$, effectively encoding the square root of the sampling density weights (metric-sensitive). Should we want to learn directly the eigendecomposition of a Laplacian, the normalized formulation of L_{norm} offers a crucial advantage. Indeed, since the first eigenvector directly encodes the sampling weights of the discrete metric of the manifold, a trained neural network predicting the eigenbasis of L_{norm} simultaneously learns both the spectral decomposition and the manifold metric in a unified manner. This eliminates the need for separate modules to predict the mass matrix, simplifying the architecture while maintaining geometric consistency. Inspired by this remarkable property for Laplacians, which generalizes beyond them (see Appendix A.2), we designed our framework to similarly learn a basis, where we decided that the first eigenvector encodes the underlying metric and from which we can explicitly compute the metric-dependent area weights.

3.2. Neural Framework for Direct Spectral Bases

Exploiting the previous insights on optimal-approximation theory and discrete Laplacians, our goal is to learn directly a basis for geometry processing on discretely sampled data of arbitrary dimensions, bypassing the need to choose a specific operator, how to discretise it, and calls to sensitive eigensolvers on its disrete approximation. Should we choose an operator, like the Laplacian, a sub-goal would be to compute its eigendecomposition directly without first discretising it (by combining mass and stiffness matrices) and post hoc eigendecomposition.

Given a point cloud $\mathcal{P}$ discretely sampling a manifold $\mathcal{M}$ with n points $\mathbf{p}_i \in \mathbb{R}^d$ for $1 \leq i \leq n$, we design a neural network $\Phi_\theta : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times K}$ to predict a K-dimensional feature vector for each point, where K is the number of basis vectors we wish to predict. These per-point predictions form a matrix $\Phi_\theta(\mathcal{P}) \in \mathbb{R}^{n\times K}$, on which we do QR decomposition, $\Phi_\theta(\mathcal{P}) = \mathbf{QR}$, where $\mathbf{Q} \in \mathbb{R}^{n\times K}$ has orthonormal columns and $\mathbf{R} \in \mathbb{R}^{K\times K}$ is upper triangular. By analogy with Laplacians, we can interpret $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_K]$, with i-th column $\mathbf{q}_i \in \mathbb{R}^n$, as the first K eigenvectors of a normalized operator, like L_{norm}, corresponding to eigenvalues $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \ldots, \lambda_K]^\top$, and ordered such that $0 = \lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_K$. For any $k \leq K$, we denote by $\mathbf{Q}_k = [\mathbf{q}_1, \ldots, \mathbf{q}_k] \in \mathbb{R}^{n\times k}$ the matrix containing the first $k \leq K$ columns of $\mathbf{Q}$.

Our pipeline computes a spectral basis directly, bypassing altogether first choosing, computing, and eigendecomposing an operator. Remarkably, we need not learn the metric M separately. Indeed, with our interpretation, the first eigenvector $\mathbf{q}_1$ directly encodes the mass matrix diagonal, by taking $M = \text{diag}(\mathbf{q}_1 \odot \mathbf{q}_1)$. We also need not learn separately the eigenvalues by exploiting the min-max theorem (Theorem 3.1) linking the eigenvalues to the maximum approximation error. During the forward pass, we generate random probe functions for the input point cloud $\mathcal{P}$ and project them onto our predicted truncated bases $\mathbf{Q}_k$ for all k. The maximum reconstruction error across these projections provides an estimate α_k, from which we compute $\lambda_{k+1} \approx \frac{1}{\alpha_k}$. This gives us eigenvalue estimates without requiring additional network parameters.

3.3. Learning by Optimal Approximations

Our framework (Fig. 1) operates on a batch of point clouds. Both at train or inference time, we progressively reconstruct probe functions by M-orthogonal projections onto the Euclidean-orthogonal estimated bases $\mathbf{Q}_k$ (see how in Appendix B.1) for increasing values of k and for each point cloud in the batch, as summarized in Algorithm 1.

Training. We train our model by learning a basis to optimally reconstruct probe functions. Depending on the choice of probe function distribution, we will be working implicitly with different operators L in the optimal reconstruc-

[1]This setting is standard. It is both common in practice for meshes, and is systematic for unstructured data like pointclouds, which is what we focus on, where potential manifold boundaries are both ill-defined and not provided. The optimal approximation theory generalizes to this case by focusing on the orthogonal of the nullspace.

Algorithm 1 Progressive Reconstruction Forward Pass

Input: Point cloud $\mathcal{P}$ with n points, number of probe functions m, number of eigenvectors K
Output: Reconstruction loss $\mathcal{L}_{\text{rec}}$ and maximum-error $e_{\max} \in \mathbb{R}^K$

Generate independently and uniformly m probe functions $\mathbf{f}^{(1)}, \mathbf{f}^{(2)}, \dots, \mathbf{f}^{(m)} \in \mathbb{R}^n$. By default, do this by iteratively applying Gaussian kernels on the k-nearset neighbor graph to independently and uniformly sampled signals of $\mathbb{R}^n$.
Compute a forward pass on the network $\Phi_\theta(\mathcal{P})$
Compute the QR-decomposition $\Phi_\theta(\mathcal{P}) = \mathbf{QR}$
for $k = 1$ to K **do**
    $\mathbf{Q}_k \leftarrow \mathbf{Q}$ truncated to the first k columns
    for $i = 1$ to m **do**
        $\mathbf{f}^{(i)}_{\text{proj},k} \leftarrow M$-projection of $\mathbf{f}^{(i)}$ onto $\mathbf{Q}_k$
        $e^{(i)}_k \leftarrow \|\mathbf{f}^{(i)} - \mathbf{f}^{(i)}_{\text{proj},k}\|_2^2$
    end for
    $i^{\max}_k \leftarrow \text{argmax}_i(e^{(i)}_k)$
    $e_{\max}.\text{append}(e^{(i^{\max}_k)}_k)$
end for
$\mathcal{L}_{\text{rec}} \leftarrow \frac{1}{mK}\sum_{i=1}^m \sum_{k=1}^K e^{(i)}_k$

tion theory (Theorems 3.1 and 3.2), leading to different estimated bases each having their own advantages. By default, we take inspiration from the Laplacian, which is the most commonly used operator, yet without computing it. Probe functions for the Laplacian should be smooth, with bounded Dirichlet energy, and uniformly distributed in this bounded set. However, computing the gradient requires the knowledge of the metric, which on unstructured data like high dimensional point clouds is not provided. We relax these constraints by generating probe functions from smoothing random functions with Gaussian kernels on the k-nearest neighbor graph of the data. The resulting distribution loosely resembles that of the constrained Laplacian, leading to similarly behaved bases, yet we neither aim for exact replica of the Laplacian nor or are we constrained to this operator or this choice of distribution.

Our training loss is based on optimal-reconstruction theory. Instead of the min-max formulation (Theorem 3.1), where for each k only one probe function in the batch (the worst approximated one) would be used to optimize the model, we switch to the equivalent PCA one (Theorem 3.2) averaging out the contribution of all probe functions. This leads to stabler optimization. Our proposed loss is thus the average reconstruction loss $\mathcal{L}_{\text{rec}}$ measuring the average quality of these progressive approximations

$$\mathcal{L}_{\text{rec}} = \frac{1}{mK}\sum_{i=1}^{m}\sum_{k=1}^{K} \|\mathbf{f}^{(i)} - \mathbf{f}^{(i)}_{\text{proj},k}\|_2^2, \tag{1}$$

where $\mathbf{f}^{(i)}, \mathbf{f}^{(i)}_{\text{proj},k} \in \mathbb{R}^n$ are the i-th probe function and its projection onto the first k estimated basis vectors. We train our model by backpropagating through the entire pipeline using $\mathcal{L}_{\text{rec}}$ as the sole training objective, enabling the network to learn the optimal basis through end-to-end gradient descent-based optimization. Note that we switched from the M-norm to the unweighted Euclidean norm. This hybrid approach, combining M-weighted projection with 2-norm error, creates an unsupervised mechanism for learning a density-related mass matrix and improves the training stability (see Appendix B.2).

Importantly, we never compute the implicit optimal reconstruction operator nor its eigenvalues during training. Nevertheless, they are easy to compute at inference time.

Inference. A feed-forward pass computes our optimal reconstruction basis for each point cloud in the inference batch. We can then easily compute the associated implicit optimal reconstruction operator or its eigenvalues for downstream geometric tasks. Thanks to the min-max formulation of optimal reconstruction theory (Theorem 3.1) the eigenvalues can be estimated from the worst-case reconstruction over all the probe functions at each spectral resolution $k \in \{1, \dots, K\}$ using $\lambda_{k+1} = \frac{1}{\max_i \|\mathbf{f}^{(i)} - \mathbf{f}^{(i)}_{\text{proj},k}\|_2^2}$. This provides theoretically-grounded eigenvalue estimates directly from the estimated basis rather than in parallel to it. Given the basis and its associated eigenvalues, we can then optionally explicitly reconstruct the implicit normalized symmetric operator by matrix multiplication $Q_K \Lambda_K Q_K^T$, or it unnormalized non-symmetric version $M^{-\frac{1}{2}} Q_K \Lambda_K Q_K^T M^{\frac{1}{2}}$. However, the operator matrix has little use in practice, as even its action is usually computed in its spectral basis. Thus, in our experiments, we never needed to recompute it explicitly. From the estimated normalized basis $\mathbf{q}_i$, we can also compute an unnormalized spectral basis $\mathbf{v}_i = M^{-\frac{1}{2}}\mathbf{q_i}$, which can be preferable downstream as it is more sampling invariant. For instance, the first vector $\mathbf{v}_1$ is constant regardless of the sampling of the the underlying smooth manifold.

4. Experiments

We demonstrate the versatility of our framework to compute spectral bases on arbitrary data. Starting from a sanity check in a toy 1D setting, with points sampled in $[0, 1] \subset \mathbb{R}$, we show that we can handle not only traditional 3D point clouds sampled on two-dimensional surfaces in $\mathbb{R}^3$, but also high-dimensional data in $\mathbb{R}^d$ such as image embeddings. Full details on the data, implementation, and further results are pushed to Appendix C.

4.1. Toy 1D Segment Manifold

As a sanity check, we illustrate our method on the simplest geometry: the unit interval $\Omega = [0, 1]$.

Figure 2. Learned eigenfunctions on [0,1] recover frequency-ordered harmonics resembling the Laplacian's spectrum.

Setting. We grid sample 100 points on $[0, 1]$, forming a point cloud. As in higher dimensions, random probe functions are first smoothened, without boundary constraints. Our learned extractor Φ_θ is a small MLP, which suffices in this single small point cloud setting.

Results. We plot in Fig. 2 the first five learned unnormalized spectral basis vectors $\mathbf{v}_i = M^{-\frac{1}{2}}\mathbf{q}_i$ after sign alignment. The network recovers a Fourier basis-like family with frequency-increasing harmonics: $\mathbf{v}_1$ is constant, while the other $\mathbf{v}_i$ exhibit i half-waves across the interval. This minimal example showcases how we can compute a frequency-ordered orthonormal basis resembling the Fourier basis, i.e. the Laplacian's eigenfunctions, just by optimising the approximation objective on basic probe functions. Importantly, this happens because the metric weights M, extracted from the first normalized basis vector $\mathbf{q}_1$, are sensible as they are correlated with intuition.

4.2. 2D Surface and 3D Volume Manifolds

We here show that our method can learn an approximation of the eigendecomposition of the most prevalent operator in shape analysis: the Laplace-Beltrami operator. By learning on single 3D point clouds (overfitting setting), we obtain highly accurate estimates. By learning on a wide collection of 3D point clouds from various datasets (generalization setting), we may obtain a foundation model that generalizes to unseen point clouds in $\mathbb{R}^3$ without retraining.

Datasets. The shapes in the overfitting setting are from [90]. In the generalization setting, we train our model on a wide collection of surface datasets to ensure broad generalization, ranging from protein structures to human scans [16, 66, 70, 75, 101]. We evaluate on reference shape analysis benchmarks [41, 48, 69, 76, 86, 90, 99, 122, 127, 146], spanning many challenges, e.g. deformations, topology variations, and different geometric characteristics. In each dataset, we dropped the mesh connectivity to work only with point clouds. Also, all shapes are scaled to fit within a unit sphere. We evaluate generalization to new manifold dimensionality by testing on 3D volumetric point clouds, computed by randomly sampling points inside the volume of shapes in [41], the model learned on surface point clouds.

Methods. We compare our neural framework (without connectivity) using a transformer [130] as learned extractor Φ_θ against the reference axiomatic oracle: the classical cotangent Laplacian (with oracle mesh connectivity).

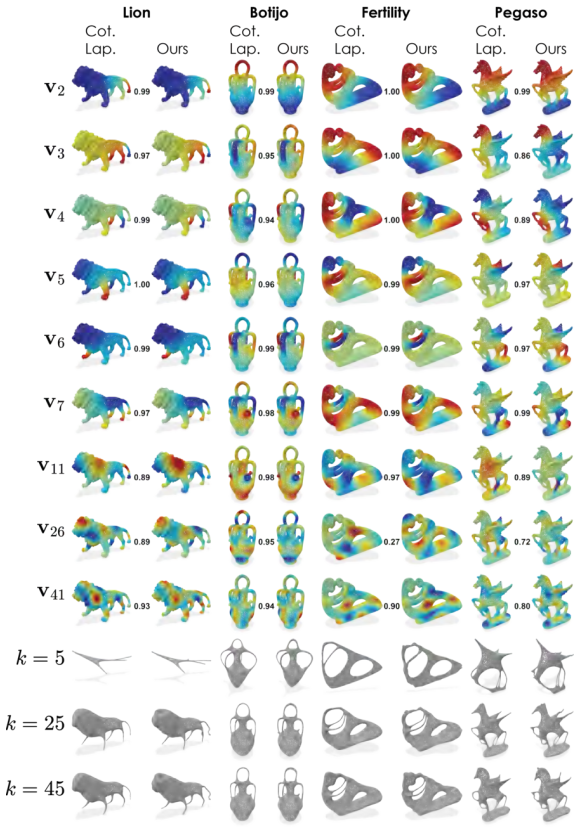


Figure 3. Unnormalized spectral basis (top) and xyz reconstruction from k basis vectors (bottom), using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors. We get similar if not more detailed reconstructions. More in Appendix C.

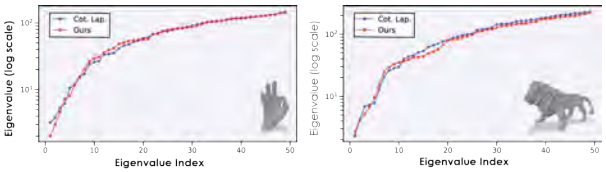


Figure 4. Eigenvalues of the oracle cotangent Laplacian and our estimated ones (overfitting setting). More in Appendix C.

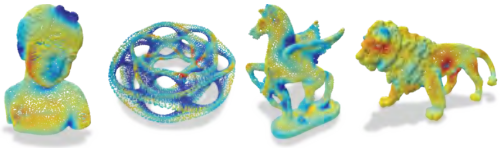


Figure 5. Estimated mass metric M from $\mathbf{q}_1$ (overfitting setting).

Results – Overfitting setting. We plot the learned unnormalized eigenvectors $\mathbf{v}_i$ in Fig. 3. Remarkably, our eigenvectors, unsupervisedly learned solely using optimal-approximation theory and without knowledge of the underlying mesh, are almost identical to those computed for the

Table 1. Average cosine similarity between predicted and oracle eigenfunctions at different truncation levels k, and mean relative eigenvalue discrepancy (overfitting setting). More in Appendix C.

Shape	Image	$k \leq 10$	$k \leq 20$	$k \leq 50$	λ Discrepancy
Armadillo		0.968	0.967	0.773	0.200 ± 0.126
Bimba		0.964	0.945	0.822	0.093 ± 0.145
Botijo		0.972	0.955	0.813	0.153 ± 0.092
Elephant		0.979	0.866	0.687	0.105 ± 0.123
Fertility		0.874	0.866	0.720	0.083 ± 0.106
Kitten		0.993	0.988	0.981	0.088 ± 0.104
Laurent Hand		0.823	0.696	0.568	0.066 ± 0.078
Lion		0.951	0.908	0.822	0.067 ± 0.086
Pegaso		0.932	0.797	0.544	0.142 ± 0.140

cotangent Laplacian, which relies on the oracle mesh structure, with cosine similarity between them often close to 1 (see Tab. 1). Additionally, the eigenvalues extracted from the worst case errors provide a good approximation to those of the oracle (see Fig. 4). These results show that our neural unsupervised method can be used in an overfitting setting to get highly accurate spectral basis estimates that match those of the targeted Laplace-Beltrami oracle. On some shapes though, e.g. Pegaso, higher frequency basis vectors slightly diverge from the oracle. Yet by analyzing shape reconstruction (spectral filtering) results (see Fig. 3), projecting the xyz coordinates to the first k spectral vectors, we see that our model captures additional details lost in the oracle, providing better top k approximation. Thus not only can we imitate the reference oracle method, we can in some cases provide a superior version with better compressed information in the spectral basis. A cornerstone of our method is the unsupervised extraction of the metric weights M directly from the first estimated normalized vector $\mathbf{q}_1$, without knowledge of the mesh structure. We see in Fig. 5 that these estimated area weights are indeed well-behaved.

Results – Generalization setting. Here, our model is trained on a wide collection of surface point clouds. We plot learned unnormalized eigenvectors $\mathbf{v}_i$ and filter the xyz coordinates on unseen evaluation shapes, either surfaces or volumes (Fig. 6), which is a type of manifold never seen in training. Our model generalizes well to new eclectic types of shapes, yet with smaller precision than in the overfitting setting, demonstrating that our framework could provide unsupervised foundation models to compute spectral decompositions generalizing well beyond the training data.

Beyond Laplacians. We also implemented our method using other families of probes – piecewise constant, smooth polynomials, and Schrödinger-like functions – corresponding to non-LBO operators, see Fig. 7. Passing the burden from the choice of operator onto that of the probe family is a favorable trade-off, as generating simple families of probes is easier than operator discretization, especially in high dimensions. See Appendix C.4.2 for full details.

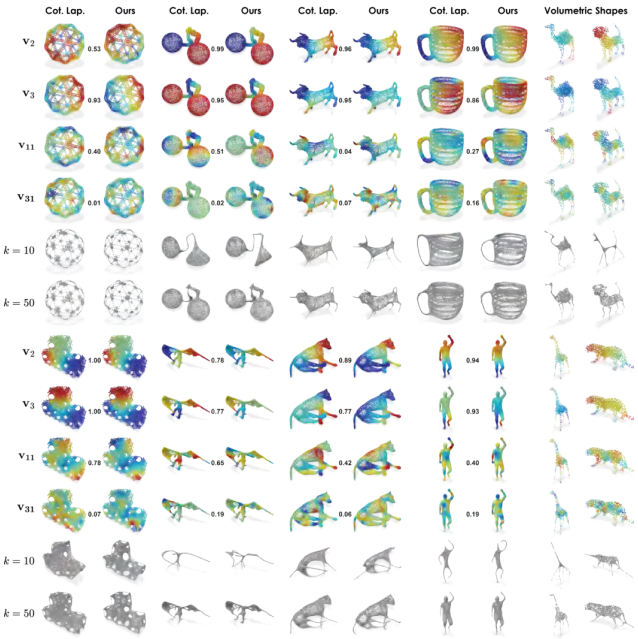
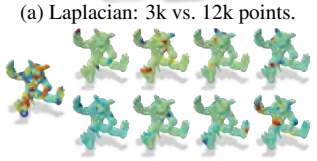
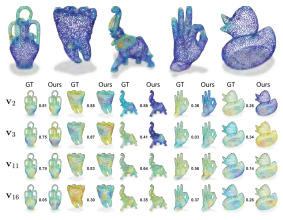


Figure 6. Unnormalized spectral basis $\mathbf{v}_1$ on unseen shapes, either surfaces (left) or volumes (right), when the model was trained on a wide collection of surface point clouds (generalization setting). Our model exhibits foundation-level generalization capabilities.

(a) Laplacian: 3k vs. 12k points.



(b) Piecewise: probe (left), basis (right).



(c) Schrödinger: potential $V(x)$ (top), learned vs. GT (bot.).

Figure 7. Results for different probe families (overfitting setting).

4.3. High-Dimensional Manifolds

To demonstrate the generality of our approach, we experiment, beyond the classical 2D surface setting, on manifolds with high intrinsic dimensionality. Here, each image is a single point on the dataset manifold, with distances between images measured in a pretrained feature space.

Datasets. We use standard image classification datasets: STL10 [34], Imagenette [59], CIFAR100 [67], and Caltech256 [53] having 10, 10, 100, and 256 classes. Here, we do not mix datasets, but follow the train-test splits. As is standard, rather than working on raw pixel images,

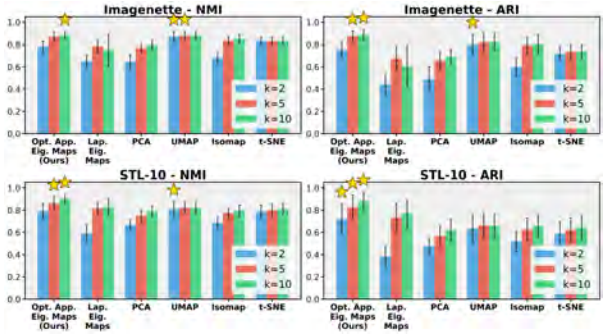


Figure 8. Average clustering performance over 50 runs of manifold learning methods on DINOv2 features of random data subsets (1500 images). Higher is better. More in Appendix C.

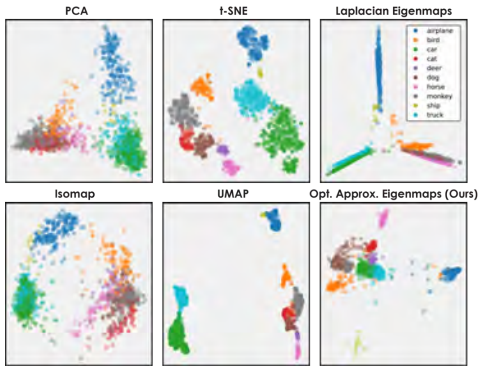


Figure 9. Manifold learning visualization of a random subset of STL10 by 2D embedding DINOv2 features. More in Appendix C.

we use reference feature embeddings, DINOv2 [92] and CLIP [104], as high-dimensional (yet lower dimensional than the raw pixel image) image coordinates in $\mathbb{R}^d$, with $d=768$ (resp. 512) for DINOv2 (resp. CLIP). Dataset manifolds are thus mapped to submanifolds of $\mathbb{R}^d$, and image distances are measured on embeddings.

Methods. Unlike 3D point clouds, high-dimensional data cannot be visualized and analyzed directly. Manifold learning addresses this by finding low-dimensional embeddings in $\mathbb{R}^k$, with $k \ll d$, that preserve pairwise dissimilarities to capture manifold structures. In particular, it enables 2D visualizations ($k = 2$) of high-dimensional data. For a short overview of manifold learning see Fig. 8. For classification data, it is widely accepted that better manifold learning methods provide better clustered embeddings correlating with the class labels. This can be both evaluated visually or with metrics such as the Normalized Mutual Information (NMI) [132] and Adjusted Rand Index (ARI) [60]. Our unnormalized spectral basis $\mathbf{v}_i$ naturally provides k-low-dimensional embeddings, which is our proposed manifold learning technique that we name Optimal-Approximation Eigenmaps. It is a direct generalization of Laplacian Eigenmaps [11], which uses the graph Laplacian spectral basis instead. We compare for various embedding dimensions $k \in \{2, 5, 10, 50\}$ our method against reference manifold learning techniques: PCA [58, 97], Isomap [126], Laplacian Eigenmaps [11], t-SNE [83, 129], and UMAP [85].

Results. Based on our results (see Figs. 8 and 9), our learned spectral basis consistently provides competitive or superior embeddings compared to baselines. These results showcase the strength of our neural spectral decomposition for capturing the intrinsic geometry of manifolds. In particular, our superiority compared to the similar Laplacian Eigenmaps suggests that our spectral basis and associated implicit operator are superior to those of the graph Laplacian, which is the standard for high-dimensional data.

5. Conclusion

We propose a neural framework to compute directly spectral bases on unstructured data of any dimensionality, bypassing the traditional need to first choose, discretize, and then eigendecompose an operator. Grounded in optimal-approximation theory, we find the orthonormal basis that optimally approximates constrained probe functions, with different probe distributions encoding different implicit operators. From the estimated basis, we can recover directly the manifold metric and compute explicitly the associated eigenvalues of the associated implicit operator. In a wide range of experiments covering one to three-dimensional manifolds, we showed that our method can recover the Laplace-Beltrami operator (LBO), which is the most useful operator in 3D geometry. However, our method scales in practice to high-dimensionality, unlike the LBO, and we show that it provides advantageous spectral representations via manifold learning experiments on image datasets. Our data-driven alternative to conventional pipelines paves the way for new avenues in geometry processing for unstructured data, especially in high-dimensions.

Limitations and future work. Training our method is computationally expensive, requiring time and GPUs, although this can be improved by optimizing our code with efficient compilation and CUDA implementations. Due to constraints, we focused on few tasks, yet we intend to explore further downstream applications, especially in high-dimensions, such as using our spectral basis in graph neural networks, or learning a more general foundation model that would apply to data of any dimensionality. At the heart of our method, the choice of probe functions imposes an underlying metric and implicit operator. By default we smoothen random functions on the kNN graph, giving similar results to the Laplacian. Yet, to best approximate it we need to manually tune distribution hyperparameters per shape, as we do in the overfitting setting. In future work, we will learn probe distributions and explore the approximation of more operators beyond the traditional Laplacians.

References

[1] Yonathan Aflalo and Ron Kimmel. Spectral multidimensional scaling. *Proceedings of the National Academy of Sciences*, 110(45):18052–18057, 2013. 5

[2] Yonathan Aflalo, Ron Kimmel, and Dan Raviv. Scale invariant geometry for nonrigid shapes. *SIAM Journal on Imaging Sciences*, 6(3):1579–1597, 2013. 2

[3] Yonathan Aflalo, Haim Brezis, and Ron Kimmel. On the optimality of shape and data representation in the spectral domain. *SIAM Journal on Imaging Sciences*, 8(2):1141–1160, 2015. 1

[4] Yonathan Aflalo, Haïm Brezis, Alfred Bruckstein, Ron Kimmel, and Nir Sochen. Best bases for signal spaces. *Comptes Rendus. Mathématique*, 354(12):1155–1167, 2016. 1, 3

[5] Yulin An and Enrique del Castillo. An ai approach for learning the spectrum of the laplace-beltrami operator. *arXiv preprint arXiv:2507.07073*, 2025. 2

[6] Mathieu Andreux, Emanuele Rodola, Mathieu Aubry, and Daniel Cremers. Anisotropic laplace-beltrami operators for shape analysis. In *European conference on computer vision*, pages 299–312. Springer, 2014. 2, 3

[7] Mathieu Aubry, Ulrich Schlickewei, and Daniel Cremers. The wave kernel signature: A quantum mechanical approach to shape analysis. *2011 IEEE International Conference on Computer Vision Workshops (ICCV Workshops)*, 1:1626–1633, 2011. 1

[8] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 3

[9] Thomas Barthelmé. A natural finsler-laplace operator. *Israel Journal of Mathematics*, 196(1):375–412, 2013. 2

[10] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps and spectral techniques for embedding and clustering. *Advances in neural information processing systems*, 14, 2001. 5

[11] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396, 2003. 8, 5

[12] Eugenio Beltrami. *Saggio di interpretazione della geometria non-euclidea*. Stab. Tip. De Angelis, 1868. 2

[13] Ido Ben-Shaul, Leah Bar, Dalia Fishelov, and Nir Sochen. Deep learning solution of the eigenvalue problem for differential operators. *Neural Computation*, 35(6):1100–1134, 2023. 2

[14] David Bensaïd, Amit Bracha, and Ron Kimmel. Partial shape similarity by multi-metric hamiltonian spectra matching. In *Scale Space and Variational Methods in Computer Vision: 9th International Conference, SSVM 2023, Santa Margherita Di Pula, Italy, May 21–25, 2023, Proceedings*, page 717–729, Berlin, Heidelberg, 2023. Springer-Verlag. 2

[15] David Bensaïd, Noam Rotstein, Nelson Goldenstein, and Ron Kimmel. Partial matching of nonrigid shapes by learning piecewise smooth functions. In *Computer Graphics Forum*, page e14913. Wiley Online Library, 2023. 2

[16] Federica Bogo, Javier Romero, Matthew Loper, and Michael J Black. Faust: Dataset and evaluation for 3d mesh registration. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3794–3801, 2014. 6, 2

[17] Davide Boscaini, Jonathan Masci, Emanuele Rodolà, and Michael Bronstein. Learning shape correspondence with anisotropic convolutional neural networks. *Advances in neural information processing systems*, 29, 2016. 2

[18] Davide Boscaini, Jonathan Masci, Emanuele Rodolà, Michael M Bronstein, and Daniel Cremers. Anisotropic diffusion descriptors. In *Computer Graphics Forum*, pages 431–441. Wiley Online Library, 2016. 2

[19] Mario Botsch, Leif Kobbelt, Mark Pauly, Pierre Alliez, and Bruno Levy. Polygon mesh processing. pages 97–122, 2010. 3

[20] Amit Bracha, Oshri Halimi, and Ron Kimmel. Shape Correspondence by Aligning Scale-invariant LBO Eigenfunctions. In *Eurographics Workshop on 3D Object Retrieval*. The Eurographics Association, 2020. 2

[21] Amit Bracha, Thomas Dagès, and Ron Kimmel. On unsupervised partial shape correspondence. In *Proceedings of the Asian Conference on Computer Vision*, pages 4488–4504, 2024.

[22] Amit Bracha, Thomas Dagès, and Ron Kimmel. Wormhole loss for partial shape matching. In *Advances in Neural Information Processing Systems*, pages 131247–131277. Curran Associates, Inc., 2024. 2, 5

[23] Matthew Brand. Nonrigid embeddings for dimensionality reduction. In *European Conference on Machine Learning*, pages 47–59. Springer, 2005. 5

[24] Haïm Brezis and David Gómez-Castro. Rigidity of optimal bases for signal spaces. *Comptes Rendus. Mathématique*, 355(7):780–785, 2017. 1, 3

[25] Alexander M Bronstein, Michael M Bronstein, and Ron Kimmel. Generalized multidimensional scaling: a framework for isometry-invariant partial surface matching. *Proceedings of the National Academy of Sciences*, 103(5): 1168–1172, 2006. 5

[26] Alexander M Bronstein, Michael M Bronstein, and Ron Kimmel. *Numerical geometry of non-rigid shapes*. Springer Science & Business Media, 2008. 2

[27] Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. Spectral networks and locally connected networks on graphs. *International Conference on Learning Representations (ICLR)*, 2013. 2

[28] Augustin-Louis Cauchy. Sur l'équation à l'aide de laquelle on détermine les inégalités séculaires des mouvements des planètes. *Oeuvres Complètes (IIeme Série)*, 9:174–195, 1829. 2

[29] Frédéric Chazal, Ilaria Giulini, and Bertrand Michel. Data driven estimation of laplace-beltrami operator. *Advances in Neural Information Processing Systems*, 29, 2016. 2

[30] Gengxiang Chen, Xu Liu, Qinglu Meng, Lu Chen, Changqing Liu, and Yingguang Li. Learning neural operators on riemannian manifolds. *National Science Open*, 3 (6):20240001, 2024. 2

[31] Yoni Choukroun, Gautam Pai, and Ron Kimmel. Sparse approximation of 3d meshes using the spectral geometry of the hamiltonian operator. *J. Math. Imaging Vis.*, 60(6): 941–952, 2018. 2

[32] Ming Chuang, Linjie Luo, Benedict J Brown, Szymon Rusinkiewicz, and Michael Kazhdan. Estimating the laplace-beltrami operator by restricting 3d functions. In *Computer graphics forum*, pages 1475–1484. Wiley Online Library, 2009. 2

[33] Fan RK Chung. *Spectral graph theory*. American Mathematical Soc., 1997. 2

[34] Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 215–223. JMLR Workshop and Conference Proceedings, 2011. 7

[35] Ronald R. Coifman and Stéphane Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1):5–30, 2006. Special Issue: Diffusion Maps and Wavelets. 1, 2, 5

[36] Ronald R Coifman, Stephane Lafon, Ann B Lee, Mauro Maggioni, Boaz Nadler, Frederick Warner, and Steven W Zucker. Geometric diffusions as a tool for harmonic analysis and structure definition of data: Diffusion maps. *Proceedings of the national academy of sciences*, 102(21): 7426–7431, 2005. 5

[37] Thomas Dagès, Michael Lindenbaum, and Alfred M Bruckstein. Metric convolutions: A unifying theory to adaptive image convolutions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 13974–13984, 2025. 2

[38] Thomas Dagès, Simon Weber, Ya-Wei Eileen Lin, Ronen Talmon, Daniel Cremers, Michael Lindenbaum, Alfred M Bruckstein, and Ron Kimmel. Finsler multi-dimensional scaling: Manifold learning for asymmetric dimensionality reduction and embedding. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 25842–25853, 2025. 5

[39] Thomas Dagès, Simon Weber, Daniel Cremers, and Ron Kimmel. Harnessing data asymmetry: Manifold learning in the Finsler world. *arXiv preprint arXiv:2603.11396*, 2026. 2, 5

[40] David L Donoho and Carrie Grimes. Hessian eigenmaps: Locally linear embedding techniques for high-dimensional data. *Proceedings of the National Academy of Sciences*, 100 (10):5591–5596, 2003. 5

[41] Roberto M Dyke, Yu-Kun Lai, Paul L Rosin, Stefano Zappalà, Seana Dykes, Daoliang Guo, Kun Li, Riccardo Marin, Simone Melzi, and Jingyu Yang. SHREC'20: Shape correspondence with non-isometric deformations. *Computers & Graphics*, 92:28–43, 2020. 6, 2, 3

[42] Yuval Eldar, Michael Lindenbaum, Moshe Porat, and Yehoshua Y Zeevi. The farthest point strategy for progressive image sampling. *IEEE transactions on image processing*, 6(9):1305–1315, 1997. 3

[43] William A Falcon. Pytorch lightning. *GitHub*, 2019. 3

[44] Bruce Fischl, Martin I Sereno, and Anders M Dale. Cortical surface-based analysis: Ii: inflation, flattening, and a surface-based coordinate system. *Neuroimage*, 9(2):195–207, 1999. 5

[45] Edward B Fowlkes and Colin L Mallows. A method for comparing two hierarchical clusterings. *Journal of the American statistical association*, 78(383):553–569, 1983. 6

[46] Clement Fuji Tsang, Maria Shugrina, Jean Francois Lafleche, Or Perel, Charles Loop, Towaki Takikawa, Vismay Modi, Alexander Zook, Jiehan Wang, Wenzheng Chen, Tianchang Shen, Jun Gao, Krishna Murthy Jatavallabhula, Edward Smith, Artem Rozantsev, Sanja Fidler, Gavriel State, Jason Gorski, Tommy Xiang, Jianing Li, Michael Li, and Rev Lebaredian. Kaolin: A pytorch library for accelerating 3d deep learning research. 3

[47] Thorben Funke, Tian Guo, Alen Lancic, and Nino Antulov-Fantulin. Low-dimensional statistical manifold embedding of directed graphs. In *8th International Conference on Learning Representations (ICLR 2020)*, pages 2018–2035. Curran, 2020. 5

[48] Daniéla Giorgi, Silvia Biasotti, and Laura Paraboschi. Watertight models track. *Shape Retrieval Contest*, 2007. 6, 2

[49] Teofilo F Gonzalez. Clustering to minimize the maximum intercluster distance. *Theoretical computer science*, 38: 293–306, 1985. 3

[50] Carolyn Gordon and David Webb. You can't hear the shape of a drum. *American Scientist*, 84(1):46–55, 1996. 1

[51] Carolyn Gordon, David L Webb, and Scott Wolpert. One cannot hear the shape of a drum. *Bulletin of the American Mathematical Society*, 27(1):134–138, 1992. 1

[52] Jianping Gou, Xia Yuan, Ya Xue, Lan Du, Jiali Yu, Shuyin Xia, and Yi Zhang. Discriminative and geometry-preserving adaptive graph embedding for dimensionality reduction. *Neural Networks*, 157:364–376, 2023. 5

[53] Gregory Griffin, Alex Holub, Pietro Perona, et al. Caltech-256 object category dataset. Technical report, Technical Report 7694, California Institute of Technology Pasadena, 2007. 7

[54] Oshri Halimi and Ron Kimmel. Self functional maps. In *2018 International Conference on 3D Vision (3DV)*, pages 710–718. IEEE, 2018. 2

[55] David K. Hammond, Pierre Vandergheynst, and Rémi Gribonval. Wavelets on graphs via spectral graph theory. *Applied and Computational Harmonic Analysis*, 30(2):129–150, 2011. 1

[56] Jiequn Han, Jianfeng Lu, and Mo Zhou. Solving high-dimensional eigenvalue problems using deep neural networks: A diffusion monte carlo like approach. *Journal of Computational Physics*, 423:109792, 2020. 2

[57] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. 3

[58] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of educational psychology*, 24(6):417, 1933. 8, 5

[59] Jeremy Howard. Imagenette: A smaller subset of imagenet, 2019. 7

[60] Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of classification*, 2(1):193–218, 1985. 8, 6

[61] Parvaneh Joharinad, Hannaneh Fahimi, Lukas Silvester Barth, Janis Keck, and Jürgen Jost. Isumap: Manifold learning and data visualization leveraging vietoris-rips filtrations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 17699–17706, 2025. 5

[62] Mark Kac. Can one hear the shape of a drum? *The american mathematical monthly*, 73(4P2):1–23, 1966. 1

[63] Jungeum Kim and Xiao Wang. Inductive global and local manifold approximation and projection. *Transactions on Machine Learning Research*, 2024. 5

[64] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 2

[65] Gustav Kirchhoff. Ueber die auflösung der gleichungen, auf welche man bei der untersuchung der linearen vertheilung galvanischer ströme geführt wird. *Annalen der Physik*, 148 (12):497–508, 1847. 2

[66] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 6, 2

[67] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 7

[68] Joseph B Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29(1):1–27, 1964. 5

[69] Zorah Lähner, Emanuele Rodolà, Michael M Bronstein, Daniel Cremers, Oliver Burghard, Luca Cosmo, Alexander Dieckmann, Reinhard Klein, Y Sahillioğlu, et al. SHREC'16: Matching of deformable shapes with topological noise. In *Eurographics Workshop on 3D Object Retrieval, EG 3DOR*, pages 55–60. Eurographics Association, 2016. 6, 2

[70] Florent Langenfeld, Tunde Aderinwale, Feryal Windal, Mahmoud Melkemi, Ekpo Otu, Reyer Zwiggelaar, David Hunter, Yonghuai Liu, Léa Sirugue, Huu-Nghia H. Nguyen, Tuan-Duy H. Nguyen, Vinh-Thuyen Nguyen–Truong, Charles Christoffer, Danh Le, Hai-Dang Nguyen, Minh-Triet Tran, Matthieu Montès, Woong-Hee Shin, Genki Terashi, Xiao Wang, Daisuke Kihara, Halim Benhabiles, Karim Hammoudi, and Adnane Cabani. SHREC 2021: Surface-based Protein Domains Retrieval. In *Eurographics Workshop on 3D Object Retrieval*. The Eurographics Association, 2021. 6, 2

[71] Thibault Lescoat, Hsueh-Ti Derek Liu, Jean-Marc Thiery, Alec Jacobson, Tamy Boubekeur, and Maks Ovsjanikov. Spectral mesh simplification. In *Computer Graphics Forum*, pages 315–324. Wiley Online Library, 2020. 2

[72] Ron Levie, Wei Huang, Lorenzo Bucci, Michael Bronstein, and Gitta Kutyniok. Transferability of spectral graph convolutional neural networks. *Journal of Machine Learning Research*, 22(272):1–59, 2021. 2

[73] Bruno Lévy. Laplace-beltrami eigenfunctions towards an algorithm that" understands" geometry. In *IEEE International Conference on Shape Modeling and Applications 2006 (SMI'06)*, pages 13–13. IEEE, 2006. 2

[74] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. Pytorch distributed: experiences on accelerating data parallel training. *Proc. VLDB Endow.*, 13(12):3005–3018, 2020. 5

[75] Yang Li, Hikari Takehara, Takafumi Taketomi, Bo Zheng, and Matthias Nießner. 4dcomplete: Non-rigid motion estimation beyond the observable surface. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12706–12716, 2021. 6, 2

[76] Z. Lian, J. Zhang, S. Choi, H. ElNaghy, J. El-Sana, T. Furuya, A. Giachetti, R. A. Guler, L. Lai, C. Li, H. Li, F. A. Limberger, R. Martin, R. U. Nakanishi, A. P. Neto, L. G. Nonato, R. Ohbuchi, K. Pevzner, D. Pickup, P. Rosin, A. Sharf, L. Sun, X. Sun, S. Tari, G. Unal, and R. C. Wilson. Non-rigid 3D Shape Retrieval. In *Eurographics Workshop on 3D Object Retrieval*. The Eurographics Association, 2015. 6, 2

[77] Ya-Wei Eileen Lin and Ron Levie. Adaptive canonicalization with application to invariant anisotropic geometric networks. In *The Fourteenth International Conference on Learning Representations*, 2026. 2

[78] Ya-Wei Eileen Lin, Ronald R Coifman, Gal Mishne, and Ronen Talmon. Hyperbolic diffusion embedding and distance for hierarchical representation learning. In *International Conference on Machine Learning*, pages 21003–21025. PMLR, 2023. 5

[79] Ya-Wei Eileen Lin, Ronald R Coifman, Gal Mishne, and Ronen Talmon. Tree-wasserstein distance for high dimensional data with a latent feature hierarchy. *arXiv preprint arXiv:2410.21107*, 2024. 5

[80] Ya-Wei Eileen Lin, Ronen Talmon, and Ron Levie. Equivariant machine learning on graphs with nonlinear spectral filters. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. 2

[81] Yongming Liu. Curvature augmented manifold embedding and learning. *arXiv preprint arXiv:2403.14813*, 2024. 5

[82] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. 3

[83] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9 (Nov):2579–2605, 2008. 8, 5

[84] Manuel Martín-Merino and Alberto Muñoz. Visualizing asymmetric proximities with SOM and MDS models. *Neurocomputing*, 63:171–192, 2005. 5

[85] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv*, 2018. 8, 5

[86] Simone Melzi, Riccardo Marin, Emanuele Rodolà, Umberto Castellani, Jing Ren, Adrien Poulenard, P Ovsjanikov, et al. SHREC'19: matching humans with different connectivity. In *Eurographics Workshop on 3D Object Retrieval*, pages 1–8. The Eurographics Association, 2019. 6, 2

[87] Mark Meyer, Mathieu Desbrun, Peter Schröder, and Alan H Barr. Visualization and mathematics III. *Mathematics and Visualization*, pages 35–57, 2003. 2

[88] Hrushikesh N Mhaskar and Ryan O'Dowd. Learning on manifolds without manifold learning. *Neural Networks*, 181:106759, 2025. 2

[89] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018. 3

[90] Ashish Myles, Nico Pietroni, and Denis Zorin. Robust field-aligned global parametrization. *ACM Transactions on Graphics*, 33(4):1–14, 2014. 6, 2

[91] Shin-ichi Ohta and Karl-Theodor Sturm. Heat flow on finsler manifolds. *Communications on Pure and Applied Mathematics: A Journal Issued by the Courant Institute of Mathematical Sciences*, 62(10):1386–1433, 2009. 2

[92] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. 8

[93] Maks Ovsjanikov, Mirela Ben-Chen, Justin Solomon, Adrian Butscher, and Leonidas Guibas. Functional maps: a flexible representation of maps between shapes. *ACM Transactions on Graphics*, 31(4):1–11, 2012. 1

[94] Gautam Pai, Alex Bronstein, Ronen Talmon, and Ron Kimmel. Deep isometric maps. *Image and Vision Computing*, 123:104461, 2022. 5

[95] Bo Pang, Zhongtian Zheng, Yilong Li, Guoping Wang, and Peng-Shuai Wang. Neural laplacian operator for 3d point clouds. *ACM Transactions on Graphics*, 43(6):1–14, 2024. 2

[96] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019. 3

[97] Karl Pearson. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin philosophical magazine and journal of science*, 2(11):559–572, 1901. 8, 5

[98] John Wilson Peoples and John Harlim. A higher order local mesh method for approximating laplacians on unknown manifolds. *arXiv preprint arXiv:2405.15735*, 2024. 2

[99] D. Pickup, X. Sun, P. L. Rosin, R. R. Martin, Z. Cheng, Z. Lian, M. Aono, A. Ben Hamza, A. Bronstein, M. Bronstein, S. Bu, U. Castellani, S. Cheng, V. Garro, A. Giachetti, A. Godil, J. Han, H. Johan, L. Lai, B. Li, C. Li, H. Li, R. Litman, X. Liu, Z. Liu, Y. Lu, A. Tatsuma, and J. Ye. SHREC'14 track: Shape retrieval of non-rigid 3d human models. In *Proceedings of the 7th Eurographics workshop on 3D Object Retrieval*. Eurographics Association, 2014. 6, 2

[100] Ulrich Pinkall and Konrad Polthier. Computing discrete minimal surfaces and their conjugates. *Experimental Mathematics*, 2(1):15–36, 1993. 1, 2

[101] Adrien Poulenard, Marie-Julie Rakotosaona, Yann Ponty, and Maks Ovsjanikov. Effective rotation-invariant point cnn with spherical harmonics kernels. In *2019 International Conference on 3D Vision (3DV)*, pages 47–56. IEEE, 2019. 6, 2

[102] Murray H Protter. Can one hear the shape of a drum? revisited. *Siam Review*, 29(2):185–197, 1987. 1

[103] B Quackenbush and PJ Atzberger. Geometric neural operators (gnps) for data-driven deep learning in non-euclidean settings. *Machine Learning: Science and Technology*, 5(4): 045033, 2024. 2

[104] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021. 8

[105] Arianna Rampini, Irene Tallini, Maks Ovsjanikov, Alex M Bronstein, and Emanuele Rodolà. Correspondence-free region localization for partial shape similarity via hamiltonian spectrum alignment. In *2019 International Conference on 3D Vision (3DV)*, pages 37–46. IEEE, 2019. 2

[106] Dan Raviv and Ron Kimmel. Affine invariant non-rigid shape analysis. *Int. J. Comput. Vision, submitted*, 2015. 2

[107] Martin Reuter, Franz-Erich Wolter, and Niklas Peinecke. Laplace–beltrami spectra as 'shape-dna' of surfaces and solids. *Computer-Aided Design*, 38(4):342–366, 2006. Symposium on Solid and Physical Modeling 2005. 1

[108] Andrew Rosenberg and Julia Hirschberg. V-measure: A conditional entropy-based external cluster evaluation measure. In *Proceedings of the 2007 joint conference on empirical methods in natural language processing and computational natural language learning (EMNLP-CoNLL)*, pages 410–420, 2007. 6

[109] Steven Rosenberg. *The Laplacian on a Riemannian manifold: an introduction to analysis on manifolds*. Number 31. Cambridge University Press, 1997. 2

[110] Guy Rosman, Michael M Bronstein, Alexander M Bronstein, and Ron Kimmel. Nonlinear dimensionality reduction by topologically constrained isometric embedding. *International Journal of Computer Vision*, 89(1):56–68, 2010. 5

[111] Conor Rowan, John Evans, Kurt Maute, and Alireza Doostan. Solving engineering eigenvalue problems with neural networks using the rayleigh quotient. *arXiv*, 2025. 2

[112] Sam T Roweis and Lawrence K Saul. Nonlinear dimensionality reduction by locally linear embedding. *science*, 290(5500):2323–2326, 2000. 5

[113] Raif M. Rustamov. Laplace-Beltrami Eigenfunctions for Deformation Invariant Shape Representation. In *Geometry Processing*. The Eurographics Association, 2007. 1

[114] Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller. Nonlinear component analysis as a kernel eigenvalue problem. *Neural computation*, 10(5):1299–1319, 1998. 5

[115] Ariel Schwartz and Ronen Talmon. Intrinsic isometric manifold learning with application to localization. *SIAM Journal on Imaging Sciences*, 12(3):1347–1391, 2019.

[116] E.L. Schwartz, A. Shaw, and E. Wolfson. A numerical solution to the generalized mapmaker's problem: Flattening nonconvex polyhedral surfaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(9):1005–1008, 1989. 5

[117] Matan Sela, Yonathan Aflalo, and Ron Kimmel. Computational caricaturization of surfaces. *Computer Vision and Image Understanding*, 141:1–17, 2015. 2

[118] Nicholas Sharp and Keenan Crane. A laplacian for nonmanifold triangle meshes. *Computer Graphics Forum*, 39 (5):69–80, 2020. 1, 2, 8

[119] Nicholas Sharp, Souhaib Attaiki, Keenan Crane, and Maks Ovsjanikov. Diffusionnet: Discretization agnostic learning on surfaces. *ACM Transactions on Graphics (TOG)*, 41(3): 1–16, 2022. 2

[120] Roger N Shepard. The analysis of proximities: multidimensional scaling with an unknown distance function. i. *Psychometrika*, 27(2):125–140, 1962. 5

[121] O Sorkine, D Cohen-Or, Y Lipman, M Alexa, C Rössl, and H P Seidel. Laplacian surface editing. *Proceedings of the 2004 Eurographics/ACM SIGGRAPH symposium on Geometry processing*, pages 175–184, 2004. 3

[122] Robert W Sumner and Jovan Popović. Deformation transfer for triangle meshes. *ACM Transactions on Graphics*, 23(3): 399–405, 2004. 6, 2

[123] Jian Sun, Maks Ovsjanikov, and Leonidas Guibas. A concise and provably informative multi-scale signature based on heat diffusion. In *Computer graphics forum*, pages 1383–1392. Wiley Online Library, 2009. 1, 2

[124] Ryota Suzuki, Ryusuke Takahama, and Shun Onoda. Hyperbolic disk embeddings for directed acyclic graphs. In *International Conference on Machine Learning*, pages 6066–6075. PMLR, 2019. 5

[125] Jian Tang, Jingzhou Liu, Ming Zhang, and Qiaozhu Mei. Visualizing large-scale and high-dimensional data. In *Proceedings of the 25th international conference on world wide web*, pages 287–297, 2016. 5

[126] Joshua B Tenenbaum, Vin de Silva, and John C Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000. 8, 5

[127] Elia Moscoso Thompson, Silvia Biasotti, Andrea Giachetti, Claudio Tortorici, Naoufel Werghi, Ahmad Shaker Obeid, Stefano Berretti, Hoang-Phuc Nguyen-Dinh, Minh-Quan Le, Hai-Dang Nguyen, Minh-Triet Tran, Leonardo Gigli, Santiago Velasco-Forero, Beatriz Marcotegui, Ivan Sipiran, Benjamin Bustos, Ioannis Romanelis, Vlassis Fotis, Gerasimos Arvanitis, Konstantinos Moustakas, Ekpo Otu, Reyer Zwiggelaar, David Hunter, Yonghuai Liu, Yoko Arteaga, and Ramamoorthy Luxman. SHREC 2020: Retrieval of digital surfaces with similar geometric reliefs. *Computers & Graphics*, 91:199–218, 2020. 6, 2

[128] Nicolás García Trillos, Chenghui Li, and Raghavendra Venkatraman. Minimax rates for the estimation of eigenpairs of weighted laplace-beltrami operators on manifolds. *arXiv preprint arXiv:2506.00171*, 2025. 2

[129] Laurens Van Der Maaten. Accelerating t-sne using tree-based algorithms. *The journal of machine learning research*, 15(1):3221–3245, 2014. 8

[130] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv*, 2017. 6

[131] Jarkko Venna, Jaakko Peltonen, Kristian Nybo, Helena Aidos, and Samuel Kaski. Information retrieval perspective to nonlinear dimensionality reduction for data visualization. *Journal of Machine Learning Research*, 11(2), 2010. 5

[132] Nguyen Xuan Vinh, Julien Epps, and James Bailey. Information theoretic measures for clusterings comparison: is a correction for chance necessary? In *Proceedings of the 26th annual international conference on machine learning*, pages 1073–1080, 2009. 8, 6

[133] Brian A Wandell, Suelika Chial, and Benjamin T Backus. Visualization and measurement of the cortical surface. *Journal of cognitive neuroscience*, 12(5):739–752, 2000. 5

[134] Yu Wang, Mirela Ben-Chen, Iosif Polterovich, and Justin Solomon. Steklov spectral geometry for extrinsic shape analysis. *ACM Transactions on Graphics (TOG)*, 38(1):1–21, 2018. 2

[135] Max Wardetzky, Saurabh Mathur, Felix Kälberer, and Eitan Grinspun. Discrete Laplace operators: no free lunch. In *Symposium on Geometry processing*, page 37. Aire-la-Ville, Switzerland, 2007. 1, 2

[136] Simon Weber, Thomas Dagès, Maolin Gao, and Daniel Cremers. Finsler-Laplace-Beltrami operators with application to shape analysis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3131–3140, 2024. 2

[137] Kilian Weinberger, Benjamin Packer, and Lawrence Saul. Nonlinear dimensionality reduction by semidefinite programming and kernel matrix factorization. In *International Workshop on Artificial Intelligence and Statistics*, pages 381–388. PMLR, 2005. 5

[138] Kilian Q Weinberger and Lawrence K Saul. Unsupervised learning of image manifolds by semidefinite programming. *International journal of computer vision*, 70:77–90, 2006. 5

[139] Aaron Wetzler, Yonathan Aflalo, Anastasia Dubrovina, and Ron Kimmel. The Laplace-Beltrami operator: a ubiquitous tool for image and shape processing. In *International Symposium on Mathematical Morphology and Its Applications to Signal and Image Processing*, pages 302–316. Springer, 2013. 2

[140] Romy Williamson and Niloy J. Mitra. Neural geometry processing via spherical neural surfaces. *Computer Graphics Forum*, 44(2):e70021, 2025. 2

[141] Oguzhan Yigit and Richard C Wilson. Lbonet: Supervised spectral descriptors for shape analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. 2

[142] Ao Zhang, Qing Fang, Peng Zhou, and Xiao-Ming Fu. Topology-controlled laplace–beltrami operator on point clouds based on persistent homology. *Graphical Models*, 139:101261, 2025. 2

[143] Zhenyue Zhang and Jing Wang. MLLE: Modified locally linear embedding using multiple weights. *Advances in neural information processing systems*, 19, 2006. 5

[144] Zhenyue Zhang and Hongyuan Zha. Principal manifolds and nonlinear dimensionality reduction via tangent space alignment. *SIAM journal on scientific computing*, 26(1): 313–338, 2004. 5

[145] Hongyu Zhou and Zorah Lähner. Laplace-Beltrami operator for gaussian splatting. *arXiv preprint arXiv:2502.17531*, 2025. 2

[146] Qingnan Zhou and Alec Jacobson. Thingi10k: A dataset of 10,000 3d-printing models. *arXiv preprint arXiv:1605.04797*, 2016. 6, 2

Learning Eigenstructures of Unstructured Data Manifolds

Supplementary Material

A. Additional Theory

A.1. PCA Alternative Formulations

Without loss of generality, assume $\mathbb{E}(f) = 0$, as is the case for our signals uniformly distributed in $\mathcal{C}_L = \{f; \|f\|_L \leq 1\}$. Denote $C = \mathbb{E}(ff^\top)$ as the covariance matrix of distributed functions f. In our case for Theorem 3.2, $C = R_L$ as $f \sim \mathcal{U}(\mathcal{S})$. The classical iterative optimization formulation for PCA is the following. PCA finds the orthonormal basis b that maximizes iteratively over k the Rayleigh quotient

$$\max_{b_k \text{ s.t. } \begin{cases} \|b_k\|_2=1 \\ b_k \perp \{b_1,\ldots,b_{k-1}\} \end{cases}} b_k^\top C b_k. \tag{2}$$

The solution to this Rayleigh optimization, and thus to PCA, is given by the eigenvectors of C in decreasing order of eigenvalues.

Another alternative formulation exists, where the PCA optimization objective is given as the iterative minimization of the expected squared approximation error, or variance, in a basis b

$$\min_{b_k \text{ s.t. } \begin{cases} \|b_k\|_2=1 \\ b_k \perp \{b_1,\ldots,b_{k-1}\} \end{cases}} \mathbb{E}\left(\left\| f - \sum_{i=1}^{k} \langle f, b_i \rangle b_i \right\|^2\right). \tag{3}$$

Indeed, let us remind the classical link between the two formulations. Denoting $B_k \in \mathbb{R}^{d \times k}$ with columns $b_1, \ldots, b_k$, with $B_k^\top B_k = I_k$, the expected squared approximation error can be rewritten as

$$\mathbb{E}\left(\left\| f - \sum_{i=1}^{k} \langle f, b_i \rangle b_i \right\|^2\right) = \mathbb{E}(\|(I - B_k B_k^\top) f\|^2) \tag{4}$$

$$= \operatorname{tr}(\mathbb{E}(f^\top (I - B_k B_k^\top)^\top (I - B_k B_k^\top) f)) \tag{5}$$

$$= \operatorname{tr}(\mathbb{E}(f^\top (I - B_k B_k^\top) f)) \tag{6}$$

$$= \operatorname{tr}(\mathbb{E}((I - B_k B_k^\top) f f^\top)) \tag{7}$$

$$= \operatorname{tr}((I - B_k B_k^\top) C) \tag{8}$$

$$= \operatorname{tr}(C) - \operatorname{tr}(B_k B_k^\top C) \tag{9}$$

$$= \operatorname{tr}(C) - \operatorname{tr}(B_k^\top C B_k) \tag{10}$$

$$= \operatorname{tr}(C) - \sum_{i=1}^{k} b_i^\top C b_i \tag{11}$$

As C is a constant, minimizing the expected squared approximation error is thus the same as maximizing the cumulative sum of Rayleigh quotients of C. Working iteratively over k, we get that the optimal solution b for both formulations is given by the eigenvectors of C in decreasing eigenvalue order. □

A.2. Normalized Operators and Eigenstructures

We here generalize the relationship between the normalized symmetric Laplacian compared to the unnormalized one regarding its first eigenvector and how it provides the metric.

For any operator A, we can define an analogous normalized operator $A_{\text{norm}} = NAN^{-1}$, where N is a matrix used for normalization. For instance $N = M^{\frac{1}{2}}$ for the normalized Laplacian L_{norm} and $A = L$. Both operators A and A_{norm} share the same eigenvalues but different eigenvectors, related by the normalization matrix N. Indeed, if λ_i, e_i^{norm} are eigenvalues and eigenvectors of A_{norm}, then $AN^{-1}e_i = N^{-1}NAN^{-1}e_i = N^{-1}A_{\text{norm}}e_i = \lambda_i N^{-1}e_i$, meaning that $e_i = N^{-1}e_i^{\text{norm}}$ are the eigenvectors of the unnormalized operator A. As such, choosing $N = M^{\frac{1}{2}}$ to be given by the metric as for the normalized Laplacian, the eigenvectors of the normalized Laplacian are related to the unnormalized one by the metric $e_i^{\text{norm}} = M^{\frac{1}{2}}e_i$. Note that A_{norm} is symmetric if $A = N^{-2}\Sigma$ for symmetric Σ and N, which is the case for the Laplacians where $\Sigma = S$ and $N = M^{\frac{1}{2}}$ is diagonal, as then $A_{\text{norm}} = N^{-1}\Sigma N^{-1}$, and so by the spectral theorem e_i^{norm} is Euclidean-orthogonal whereas e_i is $N^\top N$-orthogonal.

Of note, as for the Laplacians, if $e_1 = \mathbf{1}$ is in the nullspace of the unnormalized operator A, then $e_1^{\text{norm}} = N\mathbf{1}$ is in the nullspace of A_{norm}. Remarkably, if N is a diagonal matrix (like $M^{\frac{1}{2}}$), if the nullspace is of dimension one, and if we learn to find the nullspace by learning the eigenbasis directly, then we can compute N directly from it as $\operatorname{diag}(N) = e_i^{\text{norm}}$. For most operators of interest, the assumptions on A hold (for connected relationship graphs). This is because operators of interest are usually linear and differential, i.e. linear combinations with constant coefficients of the derivative operator $A = \sum_{|k| \leq m} \alpha_k \partial^k$, usually without a 0-th order term, and constant functions are zeroed out by derivatives.

As such, our methodology learning a Euclidean orthogonal basis directly is well-suited to capture eigenstructures of implicit operators with Euclidean orthogonal eigenvectors and a single nullspace eigenvector dimension given by a diagonally scaled version of the $\mathbf{1}$ vector. The normalized Laplacian falls into this category, yet it is just one out of the many other interesting operators satisfying these assumptions.

B. Additional Details on Our Framework

B.1. Projecting onto Optimal Reconstruction Bases

To approximate probe signals, we need to compute projections onto $\mathbf{Q}_k$. However, Euclidean, that is uniform, orthogonal projection will not account for the non-uniform geometry of the sampled manifold due to non-uniform sampling density. To overcome this issue, we resolve to the M-orthogonal projection onto the columns of $\mathbf{Q}_k$. As they are the output of a QR-decomposition, these vectors are Euclidean orthogonal and not M-orthogonal, yet, we want to M-orthogonally project onto their span. To do this, the M-orthogonally projected functions can be rewritten as

$$\mathbf{f}_{\text{proj},k} \quad = \quad \sum_{i=1}^{k} c_i^{(k)} \mathbf{q}_i \quad = \quad \mathbf{Q}_k \mathbf{c}^{(k)}, \tag{12}$$

where $\mathbf{c}^{(k)} = [c_1^{(k)}, \dots, c_k^{(k)}]^\top \in \mathbb{R}^k$ contains the projection coefficients. The coefficients $\mathbf{c}^{(k)}$ are determined by requiring that the residual $(\mathbf{f} - \mathbf{f}_{\text{proj},k})$ is M-orthogonal to all basis vectors

$$(\mathbf{f} - \mathbf{f}_{\text{proj},k})^\top M \mathbf{q}_i \; = \; 0 \quad \text{for} \; \; i \; = \; 1, \dots, k. \tag{13}$$

This orthogonality condition yields the system

$$\mathbf{q}_i^\top M \mathbf{f} \quad = \quad \sum_{j=1}^{k} c_j^{(k)} (\mathbf{q}_i^\top M \mathbf{q}_j). \tag{14}$$

In matrix form, this becomes

$$\mathbf{G}^{(k)} \mathbf{c}^{(k)} \quad = \quad \mathbf{b}^{(k)}, \tag{15}$$

where $\mathbf{G}^{(k)} \in \mathbb{R}^{k \times k}$ is the Gram matrix with entries $G_{ij}^{(k)} = \mathbf{q}_i^\top M \mathbf{q}_j$, and $\mathbf{b}^{(k)} \in \mathbb{R}^k$ is a vector with entries $b_i = \mathbf{q}_i^\top M \mathbf{f}$. Since $\mathbf{G}^{(k)} = \mathbf{Q}_k^\top M \mathbf{Q}_k$ is symmetric positive definite[2], we can efficiently solve this linear system using a differentiable solver to obtain the coefficients $\mathbf{c}^{(k)}$[3]. Crucially, $\mathbf{G}^{(k)} \in \mathbb{R}^k \times k$ is small, where $k \ll n$ is the number of eigenvectors used for projection, making the system computationally efficient and independent of the input size n. Note that $\mathbf{c}^{(k)}$ is not the truncation of $\mathbf{c}^{(K)}$ to its first k terms since $\mathbf{Q}_k$ is not orthogonal for the used M-weighted scalar product. These independent systems are batched and solved concurrently for all $k \leq K$.

B.2. Replacing the M-norm with the Euclidean norm for Approximation Errors

While the theoretical framework calls for the M-norm in the error computation (in both Theorems 3.1 and 3.2), we deliberately use the 2-norm in our loss (Eq. (1)). This hybrid approach, using M-weighted projection with 2-norm error, creates an unsupervised mechanism for learning a density-related mass matrix. Since the 2-norm weights all points equally regardless of sampling density, regions with different sampling densities contribute differently to the loss, implicitly encouraging the network to learn mass elements connected to the ambient space's Euclidean-induced volume (or area on surface manifolds) of the samples. We also experimented with using the M-norm for error computation as prescribed by theory. However, this resulted in unstable training, numerical issues (NaNs), and even collapses of the metric to 0 to naively minimise $\mathcal{L}_{\text{rec}}$. This confirms the necessity of our hybrid approach.

[2] As the columns of $\boldsymbol{Q}_k$ are linearly independent thanks to the QR-decomposition and M is positive definite.

[3] Avoiding the explicit inversion in the formula $\mathbf{c} = (\mathbf{G}^{(k)})^{-1}\mathbf{b}$.

C. Additional Experimental Details and Results

C.1. Toy 1D Segment Manifold

In this toy setting we work on a single point cloud. We uniformly grid sample $n = 100$ points $x_i = \frac{i}{n-1}$ for $i = 0, \dots, n-1$. Probe signals $\mathbf{f}_i$ are generated by drawing i.i.d. noise $\mathbf{g}_i \in \mathbb{R}^n$ and applying a few steps of 1D Gaussian smoothing on the kNN graph with 16 neighbors, yielding smooth probes $\mathbf{f}_i$. Our lightweight MLP consists in 3 hidden layers with width 64 and ReLU activations. We only compute the first $K = 5$ basis vectors. Wet train over batches of $m = 500000$ probe functions using the Adam [64] optimizer with an initial learning rate of $\eta = 10^{-2}$, a step scheduler with parameter $\gamma = 0.1$ at 30% and 70%.

C.2. 2D Surface and 3D Volume Manifolds

C.2.1. Datasets

Overfitting setting. We use shapes from [90] commonly used for benchmarking geometry processing works. We discard the mesh connectivity to work with point clouds.

Generalization setting. In order to train a foundation model for estimating spectral bases on 3D pointclouds, we unsupervisedly trained our model on many datasets covering a large spectrum of types of shapes: SURREAL [101] for synthetic human bodies, SHREC'21 Protein Domains [70] for biological structures, ABC [66] for CAD models, FAUST [16] for real human scans, and DeformingThings4D [75] for dynamic non-rigid surfaces. Evaluation is performed on well-established evaluation benchmarks covering many challenging settings: DefTransfer [122], MPZ14 [90], SHREC'07 Watertight [48], SHREC'14 Human Models [99], SHREC'15 Non-Rigid [76], SHREC'16 Topological Noise (TopKids) [69], SHREC'19 Humans [86], SHREC'20 Non-Isometric (SHREC'20-NI) [41], SHREC'20 Geometric Reliefs (SHREC'20-GR) [127], and Thingi10K [146]. All

shapes in the datasets, both in train and evaluation, originally come with mesh connectivity. When using our method, we remove all this oracle connectivity information to keep only unstructured point clouds. We also evaluate, but not train, on volumetric point clouds generated from surface meshes in SHREC'20 Non-Isometric [41]. To generate this data, we perform volumetric sampling using Kaolin's [4] [46] GPU-accelerated approach. Random candidate points are uniformly sampled within the mesh's axis-aligned bounding box (with 5% padding), then for each point we compute the unsigned distance to the nearest mesh surface and determine inside/outside classification via ray casting[5]. Points classified as interior are retained and subsampled to the target count, optionally combined with surface vertices to form the final volumetric point cloud.

C.2.2. Implementation – Generalization Setting

The following implementation details describe our generalization model that learns to predict a spectral basis resembling Laplacian eigenfunctions across diverse 3D shapes, rather than overfitting to a single geometry. The model is trained on a large-scale dataset combining multiple datasets to generalize to unseen shapes at test time.

Network Architecture. Our 3D model processes point clouds through a three-stage pipeline. First, raw 3D coordinates are passed through a preprocessing MLP with architecture [3 $\rightarrow$ 64 $\rightarrow$ 256 $\rightarrow$ 1024] using GELU activations [57] and layer normalization [8]. The resulting 1024-dimensional features are then processed by an 8-layer Transformer encoder with 32 attention heads, where each point attends to all other points in the point cloud via global self-attention. The Transformer uses $d_{\text{model}} = 1024$, feedforward dimension of 1024, GELU activations, and processes sequences in batch-first format. Finally, a lightweight output MLP [1024 $\rightarrow$ 256 $\rightarrow$ 50] with GELU activations and layer normalization predicts the 50 feature vectors for each point (prior to QR decomposition to get the estimated normalized spectral basis).

Training Configuration. We optimize with AdamW [82] (learning rate 5×10^{-4}) and cosine annealing schedule decaying to 1×10^{-5} over 100 epochs. We apply gradient clipping at 0.01 and accumulate gradients over 2 batches. The training set consists of approximately 200000 3D shapes (file sizes 0.2-7MB) from diverse datasets with a batch size of 30.

Data Processing. Each mesh is sampled to 1500 points using Farthest Point Sampling (FPS) [42, 49], with random initialization during training and fixed initialization during validation to ensure reproducibility. We generate 500 scalar fields per shape by sampling uniform random values in the range [-1, 1] at each vertex. Before computing the reconstruction loss, scalar fields are smoothed for 20 iterations using Gaussian kernel convolution with $\sigma = 0.1$ over a k-nearest neighbor graph with $k = 15$. The kNN graph construction uses Euclidean distance (not cosine similarity) and includes self-loops. Oracle cotangent Laplacian eigendecompositions are precomputed and cached to accelerate the validation phase, where we monitor cosine similarity between predicted and oracle eigenvectors.

Computational Resources. All experiments were conducted on 8 GPUs (type NVIDIA A100-SXM4-40GB) with automatic mixed precision [89] training enabled through PyTorch Lightning [43, 96].

C.2.3. Implementation – Overfitting setting

The following implementation details describe the configuration for overfitting the eigendecomposition of a specific 3D shape. This setup employs a smaller architecture to achieve maximum accuracy for a single target geometry.

Network Architecture. The architecture for single-shape overfitting uses a more compact design compared to the generalization model. Raw 3D coordinates are processed through a preprocessing MLP with architecture [3 $\rightarrow$ 32 $\rightarrow$ 128 $\rightarrow$ 256] using GELU activations and layer normalization. The core network consists of an 8-layer Transformer encoder with 8 attention heads (compared to 32 for generalization), $d_{\text{model}} = 256$, and feedforward dimension of 256. The output head is a simple two-layer MLP [256 $\rightarrow$ 50] with GELU activation and layer normalization that directly predicts the 50 Laplacian eigenvectors.

Training Configuration. We optimize with AdamW (learning rate 0.001) and cosine annealing schedule decaying to 1×10^{-5} over 5000 epochs, training for a total of 12,000 epochs. Gradient clipping is applied at 0.01. The training consists of a single mesh with batch size 1, allowing the model to specialize completely to the target geometry.

Data Processing. The single training mesh is sampled to 4000 points using Farthest Point Sampling (FPS) with random initialization at each iteration to provide variation during training. We use a higher point count compared to the generalization setting (4000 vs 1500) since batch size 1 requires significantly less GPU memory, allowing for denser point sampling. We generate 1500 scalar fields per shape by sampling uniform random values in the range [-1, 1] at each vertex. Before computing the reconstruction loss,

[4] https://github.com/NVIDIAGameWorks/kaolin

[5] Shooting a ray and counting mesh intersections, with odd counts indicating interior points.

each scalar field is smoothed for 40 iterations using Gaussian kernel convolution over a k-nearest neighbor graph with $k = 10$, where σ is randomly sampled from the range [0.01, 0.2] independently for each scalar field. We employ this range of smoothing scales rather than a fixed value because we empirically observed improved results across various shapes. We speculate that sampling different σ values allows the network to sense the shape's manifold at multiple scales, which helps generate a statistically sufficient number of smooth functions that are smooth with respect to the surface metric – a requirement for optimal approximation. Empirically, we found that using a range of σ values does not contribute significantly to the generalization case, suggesting this multi-scale sensing is particularly beneficial when overfitting to a specific geometry. The kNN graph uses Euclidean distance and does not include self-loops. Validation is performed every 200 epochs, with model checkpoints saved at the same frequency.

Computational Resources. Experiments were conducted on a single NVIDIA GPU using PyTorch Lightning. The compact architecture and small batch size make this configuration accessible for consumer-grade hardware such as an RTX 3090, requiring significantly fewer computational resources than the generalization model.

Eigenvalues at Inference Time. Our model is designed to compute a spectral basis. However, we can recover without additional cost as a byproduct the associated eigenvalues of the implicit optimal-reconstruction operator. It is given by the invert of the worse approximation error. Depending on how we generate probe functions at inference time, we get different errors and thus different eigenvalues. Our methodology to generate probe functions depends on three hyperparameters: the number of nearest neighbors k in the kNN graph, the number of smoothing iterations, and the smoothing scale σ. By default, we propose to take at inference time the fixed hyperparameters $k = 70$, 48 iterations, and $\sigma = 0.101$. These numbers gave us the minimal mean relative eigenvalue discrepancy with the oracle cotangent Laplacian's eigenvalues across all tested shapes. However, we also manually tuned these hyperparameters per shape and provide in Tab. 2 the best ones we found. Unless specified otherwise, presented results used the tuned version rather than the fixed one. In future work we intend to provide an automatic mechanism for finding the best hyperparameters per shape. For instance, as probe functions depend differentiably on σ, it could be learned via descent strategies.

C.2.4. Additional Results

Due to page-length constraints, we here provide additional results to those in the main paper.

Table 2. Optimal hyperparameters for generating probe functions in the single-shape overfitting setting when estimating eigenvalues. For each shape, we report the k-nearest neighbors (k), number of smoothing iterations, and Gaussian kernel bandwidth (σ) that yielded the best alignment with the oracle cotangent Laplacian's eigenvalues.

Shape	k	**Iterations**	σ
Armadillo	45	111	0.194
Beetle	70	120	0.120
Bimba	33	57	0.101
Botijo	27	93	0.120
David	33	48	0.138
Dente	21	75	0.101
Dragon	70	48	0.269
Elephant	57	75	0.157
Eros	27	102	0.138
Fertility	63	40	0.176
Heptoroid	39	40	0.213
Horse	70	75	0.194
Kitten	27	66	0.120
Knot	15	66	0.101
Laurent Hand	21	102	0.101
Lion	39	111	0.176
Master Cylinder	15	102	0.269
Pegaso	70	120	0.213
Teddy	45	48	0.138
Woodenfish	70	120	0.232
Wrench	70	120	0.082

Overfitting setting. In Figs. 11 to 14, we provide additional visualizations of the learned unnormalized spectral basis $\mathbf{v}_i$ and obtained spectral filtering, generalizing Fig. 3 to other shapes. We also plot on a few shapes in Figs. 15 to 19 all the first 50 spectral basis vectors (excluding the first constant one $\mathbf{v}_1$), i.e. $\mathbf{v}_2, \dots, \mathbf{v}_{50}$. In Fig. 21, we plot eigenvalue curves on other shapes than in Fig. 4 in the tuned hyperparameter setting for generating inference probe functions, along with those obtained in the fixed hyperparameter setting in Fig. 22. In Fig. 20, we display more estimated metrics M from $\mathbf{q}_1$ on many more shapes than in Fig. 5. In Tab. 3, we give further quantitative results between our estimated eigendecomposition and the oracle cotangent one[6] for the average cosine similarity and the eigenvalue discrepancy between both bases, extending Tab. 1. These results supplement those in the main manuscript demonstrating how well we are able to recover the oracle Laplacian spectral decomposition, both the basis and associated eigenvalues, with our direct neural method without any knowledge on the underlying mesh structure. While extremely

[6]Using the ground-truth mesh information that our point cloud method does not have access to.

Table 3. Average cosine similarity between predicted and oracle eigenfunctions at different truncation levels k, and mean relative eigenvalue discrepancy (overfitting setting).

Shape	Image	$k \leq 10$	$k \leq 20$	$k \leq 50$	λ Discrepancy
Beetle		0.855	0.657	0.450	0.514 ± 0.706
David		0.803	0.879	0.823	0.098 ± 0.126
Dente		0.981	0.976	0.910	0.106 ± 0.150
Dragon		0.879	0.827	0.529	0.197 ± 0.481
Eros		0.952	0.891	0.640	0.083 ± 0.156
Heptoroid		0.796	0.757	0.509	0.182 ± 0.185
Horse		0.927	0.908	0.718	0.107 ± 0.128
Knot		0.536	0.687	0.504	0.127 ± 0.279
Wrench		0.938	0.817	0.476	0.694 ± 0.222
Master Cylinder		0.948	0.923	0.779	0.051 ± 0.045
Teddy		0.987	0.981	0.912	0.109 ± 0.146

similar at low frequencies, differences emerge at higher frequencies, revealing that our method can account for different details from the oracle, while still preserving the same overall (i.e. low-pass) information of the shape manifold. Unlike the eigenfunctions, recovering the same eigenvalues as those of the oracle Laplacian is trickier and often requires carefully tuning hyperparameters to generate the appropriate probe functions at test time.

Generelization setting. In Tab. 5, we provide quantitative results comparing our predicted and oracle eigenfunctions on each evaluation dataset. Our generalization method quantitatively demonstrates potential for designing a foundation model to estimate Laplacian spectral bases. A proper foundational model would be achieved by greatly scaling up the amount of training data, as is usually done for foundation models in other fields like image or text processing. In our limited training setting, yet still large for initial insights, our model is able to accurately estimate the first Fourier eigenfunctions, which corresponds to low-pass information and thus to the most important global information on the manifold. As expected, performance is naturally lower than in the single-shape overfitting setting.

C.3. High-Dimensional Manifolds

C.3.1. Short Manifold Learning Overview

Manifold learning is a classical task [44, 68, 120, 133] in data analysis and computer vision. It consists in finding low-dimensional representations capturing the manifold structure of high-dimensional data. In practice, methods aim to find very low-dimensional embeddings in $\mathbb{R}^k$, with $k \ll d$ (where d is the original high dimension), by preserving pairwise dissimilarities based on distances. The wide collection of approaches is usually grouped based on whether the targeted preserved structures of the manifold are local [10, 11, 35, 36, 40, 52, 78, 79, 81, 84, 85, 112, 124, 125, 131, 143, 144], global [23, 47, 137, 138], or a combination of both [1, 22, 25, 38, 39, 58, 61, 63, 83, 94, 97, 110, 114–116, 126].

C.3.2. Implementation

For manifold learning experiments on image datasets, we work on the embedded feature manifold provided by pretrained vision models. Thus, each image is mapped to a high-dimensional[7] feature space, and we learn the spectral decomposition of the resulting manifold structure across four benchmark datasets: Caltech256, CIFAR100, Imagenette, and STL-10.

Feature Extraction. We experiment with two pretrained vision models as feature extractors: CLIP[8] producing 512-dimensional embeddings, and DINOv2[9] producing 384-dimensional embeddings. These frozen feature extractors map images to points on a learned manifold, where geometric relationships reflect semantic similarities. The extracted embeddings are used directly as spatial coordinates for kNN graph construction with cosine similarity.

Network Architecture and Hyperparameters. In Tab. 4, we detail the architecture and probe function generation hyperparameters for each experiment. For CLIP features, we use a preprocessing MLP [512 $\rightarrow$ 256], followed by a 6-layer Transformer encoder with 8 attention heads, $d_{\text{model}} = 256$, feedforward dimension of 256, dropout of 0.1, and GELU activations. The output head is [256 $\rightarrow$ 64 $\rightarrow$ 11] with layer normalization. For DINOv2 features, we use a more compact architecture: preprocessing MLP [384 $\rightarrow$ 128], 6-layer Transformer with 4-8 attention heads depending on the dataset, $d_{\text{model}} = 128$, and a simpler output head [128 $\rightarrow$ 11].

Training Configuration. We optimize with AdamW (learning rate 0.0005) and accumulate gradients over 16 batches. Gradient clipping is applied at 0.01. Training uses distributed data parallel (DDP) [74] strategy across multiple GPUs. Each dataset samples 5000 images uniformly, with deterministic sampling for validation and random sampling for training. We predict 11 eigenvectors for spectral clustering evaluation at $k \in \{2, 5, 10\}$. We generate 300 probe functions per manifold by sampling uniform random values in [-1, 1] at each point.

[7] Yet lower-dimensional than the original image dimensions.

[8] https://huggingface.co/openai/clip-vit-base-patch32

[9] https://huggingface.co/facebook/dinov2-small

Evaluation Setting. We compare our method against classical manifold learning approaches including PCA, UMAP, t-SNE, Isomap, and Laplacian Eigenmaps. For evaluation, we use class-weighted sampling to create challenging, non-uniform manifold distributions. Specifically, we sample 1500 images using random class weights with entropy in the range [0.01, 0.1], ensuring that the sampled manifolds have imbalanced class distributions. This sampling strategy tests the robustness of each method to realistic, non-uniform data distributions across image classes. Due to the randomness of the sampling and of some of the methods, including ours due to random probe functions, we perform 32 runs for each method on each dataset.

Sensitivity to Hyperparameters. Similar to the single-shape overfitting experiments in the 3D case, we observe that results in the image manifold setting are sensitive to the choice of probe function smoothing hyperparameters (k, σ, iterations). The parameters in Table 4 were tuned per dataset to achieve good performance. In future work, these probe function parameters could potentially be learned during training rather than manually tuned. We emphasize that the current results showcase the potential of our method, but more optimized tuning should yield better results. As such, we believe the full potential of our approach for image manifold learning has yet to be realized.

Computational Resources. All experiments were conducted on 8 GPUs (type NVIDIA A100-SXM4-40GB) using PyTorch Lightning with DDP training strategy.

Table 4. Architecture and probe function generation hyperparameters for image manifold experiments. All experiments use 6 Transformer layers, 11 predicted eigenvectors, 5000 sampled images, and 300 probe functions. The kNN graph always uses cosine similarity with self-loops enabled.

Dataset	Feature Extractor	d_{model}	Attn. Heads	k	σ	Iterations
Caltech256	CLIP	256	8	10	0.04	20
Caltech256	DINOv2	128	8	15	0.15	40
CIFAR100	CLIP	256	8	5	0.04	20
CIFAR100	DINOv2	128	4	30	0.20	30
Imagenette	CLIP	256	8	20	0.08	20
Imagenette	DINOv2	128	4	30	0.10	30
STL-10	CLIP	256	8	35	0.08	20
STL-10	DINOv2	128	4	20	0.10	20

C.3.3. Results

We evaluate our method on four image datasets (Caltech256, CIFAR100, Imagenette, and STL-10) using both CLIP and DINOv2 embeddings, comparing against classical manifold learning methods: PCA, UMAP, t-SNE, Isomap, and Laplacian Eigenmaps. Results are reported for spectral clustering with $k \in \{2, 5, 10\}$ clusters across seven standard clustering metrics: Normalized Mutual Information (NMI) [132], Adjusted Rand Index (ARI) [60], Completeness [108], Adjusted Mutual Information (AMI) [132], Homogeneity [108], V-Measure [108], and Fowlkes-Mallows Index (FMI) [45].

Quantitative Results. In Tabs. 6 to 13 we present comprehensive results, averaged over our 32 runs, across all datasets and feature extractors. Note that UMAP and t-SNE often get better clustering scores, which is unsurprising as these methods are designed to handle classification datasets by overclustering their embeddings (better scores yet often artificial separation between clusters). Our method (Optimal Approximation Eigenmaps) demonstrates competitive performance across most settings. Our approach generally outperforms classical methods like PCA, Isomap, and most importantly, shows superior performance to its vanilla analog, Laplacian Eigenmaps, in most cases. On CIFAR100 with CLIP embeddings, our method achieves particularly strong results at $k = 10$ (NMI $= 0.641$, ARI $= 0.257$), matching or exceeding Laplacian Eigenmaps while substantially outperforming PCA and Isomap. Similarly, on Caltech256 with DINOv2 features at $k = 5$, we achieve NMI $= 0.714$ and ARI $= 0.246$, demonstrating effective learned spectral representations. The method shows consistent improvements as the number of clusters increases from 2 to 10, suggesting that higher-dimensional embeddings better capture the manifold structure.

Qualitative Analysis. In Figs. 23 to 30, we plot the 2D (i.e. $k = 2$) clustering results for Imagenette and STL-10 datasets with both CLIP and DINOv2 embeddings. These visualizations demonstrate that our learned spectral basis captures meaningful semantic structure in the image manifolds, with distinct clusters corresponding to different object categories that are better clustered than in traditional methods like PCA and Isomap. In addition to accurate clusters, our embeddings provide smooth transitions between clusters, indicating that the learned basis successfully preserves the underlying smooth manifold geometry, unlike methods artificially overclustering the data like t-SNE and UMAP. This important observation demonstrates that our method is in fact superior to modern references t-SNE and UMAP, which cluster very well as revealed by the quantitative results but poorly preserve the manifold structure. Our visualisations are significantly better than those of Laplacian Eigenmaps, the vanilla analog of our method using the Graph Laplacian eigenfunctions instead of our Optimal Approximation Operator's eigenfunctions. Indeed, in Laplacian Eigenmaps clusters are ill-shaped and artificially separated unlike in our method.

C.4. Generality, adaptivity, and sensitivity of our method

C.4.1. Point cloud size

We emphasize that our method is not constrained to a fixed point cloud size. Indeed, our transformer, which forms the backbone of the method, naturally handles variable-length sequences. While we used a fixed size during training to get efficient batching, the model generalizes to different point counts at inference. For instance, for 3D data, the same overfitted Pegaso model (trained on 4k points) achieves mean cosine similarity 0.606 at 3k points and 0.661 at 12k points. See Fig. 7a for the plot of the 12th basis function inferred on both point clouds of the same Pegaso shape. The adaptivity to sample size also holds for high-dimensional manifolds. Indeed, our results in Figs. 8 and 9 were achieved on models trained on 5000 image samples yet evaluated on 1500. As such, the generalization and robustness to point cloud size at inference holds across all tested shapes and data.

C.4.2. Probe function family

Our default probe function distribution is given by smooth functions obtained by Gaussian smoothing of random functions on the k-nearest neighbor graph. This distribution yields spectral bases resembling those of the Laplacian, which is the default operator in spectral analysis. Nevertheless, our method is not constrained to this probe distribution nor this operator. We show how the resulting spectral basis evolves when working with other probe distributions.

Piecewise-constant probes. To mimic semantic priors, e.g. part labels, we defined probes as random piecewise-constant functions on K clusters. We observe two regimes. When predicting fewer spectral functions than clusters ($k < K$), the network converges to a Walsh-Hadamard-like basis (see Fig. 7b). However, when $k = K$, the spectral basis converges to cluster indicator functions.

Polynomial probes. When the probes are taken to be random 3D-multivariate polynomials of degree at most 2, which is a 10-dimensional space spanned by $\{1, X, Y, Z, X^2, Y^2, Z^2, XY, XZ, YZ\}$, the network converges to a basis spanning the same polynomial subspace (99.93% variance explained vs. 0.24% for a random baseline), despite polynomials being smooth like Laplacian eigenfunctions. As such, smoothness-only is not sufficient to get a spectral basis resembling that of the Laplacian, as we also crucially require that the distribution of smooth functions to be uniform. Unlike uniformly random polynomials, Gaussian-smoothed uniformly random probes approximately uniformly sample from the Laplacian's low-energy eigenspace.

Schrödinger-like probes. When modulating probes by a potential proportional to Gaussian curvature before smoothing, the recovered basis resembles eigenfunctions of the Hamiltonian $H = \Delta + \beta V$ (Fig. 7c), which is the Laplacian's natural generalization with a potential.

These experiments confirm that different probe families yield different spectral bases, each having their own properties and thus advantages for downstream tasks. Our default probe distribution gives a Laplacian-like spectral basis, which is the most useful family of basis in general for spectral analysis. Additionally, designing families of probe distributions is a fairly easy task when compared to discretizing operators, especially in high-dimensions. Passing the burden from the choice of operator to that of the distribution of probes is thus a significant advantage.

C.4.3. Hyperparameter sensitivity

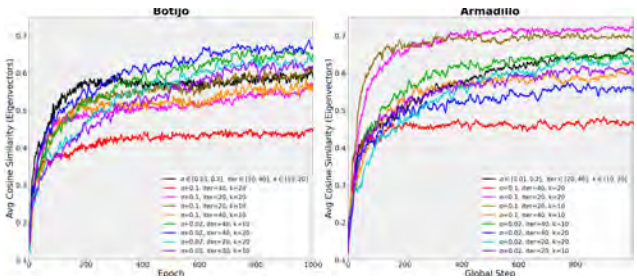


Figure 10. Hyperparameter sensitivity.

Our method does not necessarily require precise hyperparameter tuning of the probe distribution. Indeed, in our 3D experiments we use identical parameters across all shapes. Sensitivity primarily appears for recovering specific eigenvalues, but not for estimating the spectral basis. In Fig. 10, we show that different hyperparameters all yield bases closely resembling cotangent Laplacian eigenfunctions. In fact, a "wildcard" setting, with randomly sampled hyperparameters per probe, achieves competitive cosine similarity. This demonstrates that there is no need for precise hyperparameter tuning when estimating the spectral basis. In Tab. 2, we show that per-shape hyperparameter tuning is required to recover exactly the cotangent Laplacian eigenvalues. Without tuning, the basis still exhibits correct spectral decay (Fig. 22) and may benefit downstream tasks where exact recovery is unnecessary, although we did not test this. Note that probe hyperparameters could also be made learnable with respect to downstream applications.

C.5. Computational cost

In the 3D overfitting setting, it takes approximately 8min with an RTX 3090 GPU to train the model on a shape with 4k points. As overfitted models generalize to other samplings of the same shape with different number of samples (see Appendix C.4.1), it is not necessary to train an overfitting model on many more points if the point cloud is larger. For inference, computing the first 50 spectral basis vectors takes 0.44s/1.14s/4.20s on 3k/12k/24k

points, which is comparable to the robust Laplacian [118] (0.20s/1.24s/2.62s). We did not use an advanced optimized implementation, such as torch.compile, flash attention, or custom CUDA kernels. Thus, inference time could be substantially improved, especially with batching. In the 3D generalization setting, it takes 24h on 8 A100 GPUs to train the model, albeit the model is only trained once (unlike in the overfitting scheme where it must be retrained for each shape). Inference is then simply a single forward pass, identical to the overfitting setting (in particular it can be done on single commercial GPU like the RTX 3090). For high-dimensional manifolds, we use small manifolds of approximately 1500 samples to run on a single RTX 3090 GPU.

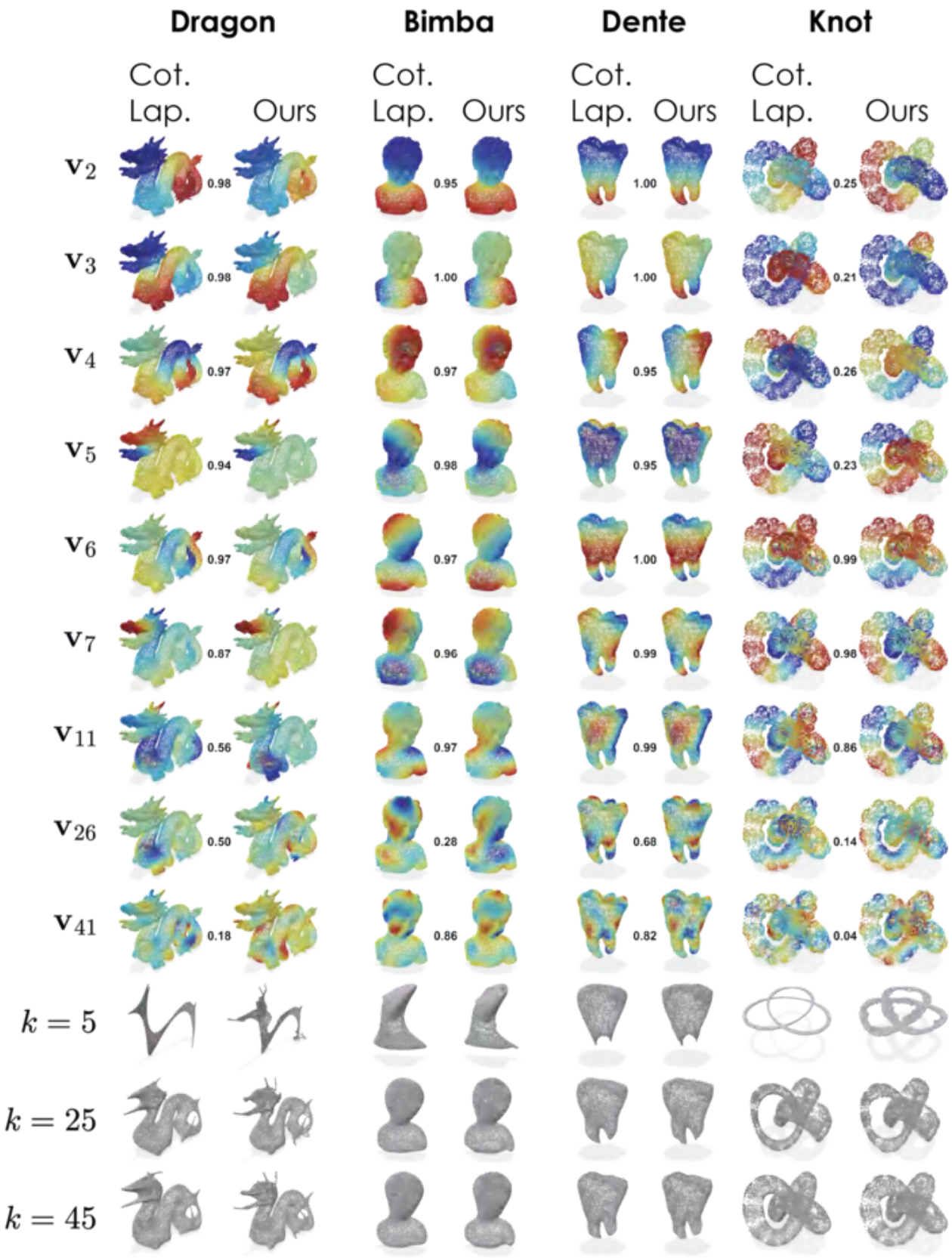


Figure 11. Unnormalized spectral basis (top) and xyz reconstruction from k basis vectors (bottom), using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

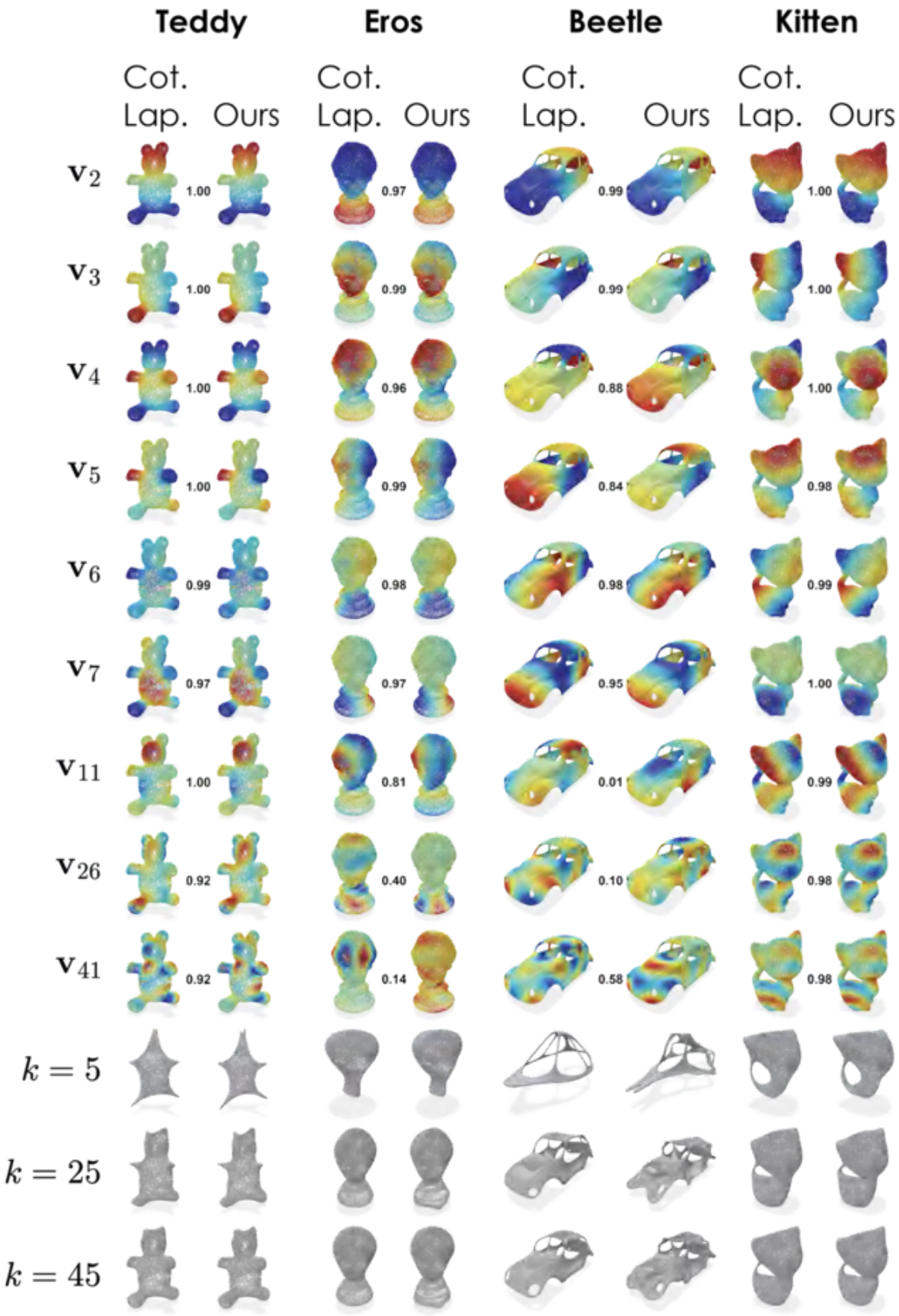


Figure 12. Unnormalized spectral basis (top) and xyz reconstruction from k basis vectors (bottom), using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

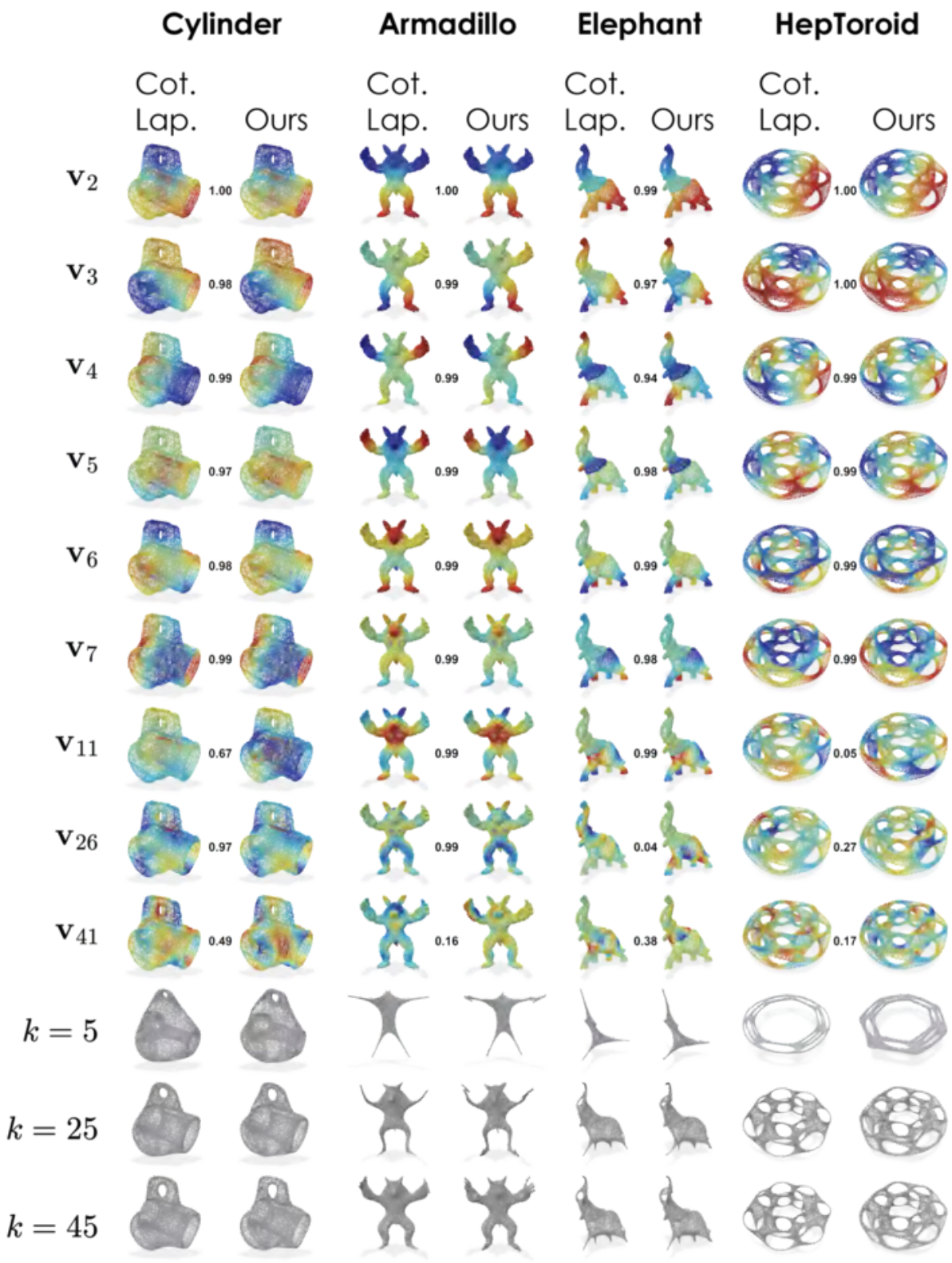


Figure 13

Unnormalized spectral basis (top) and xyz reconstruction from k basis vectors (bottom), using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

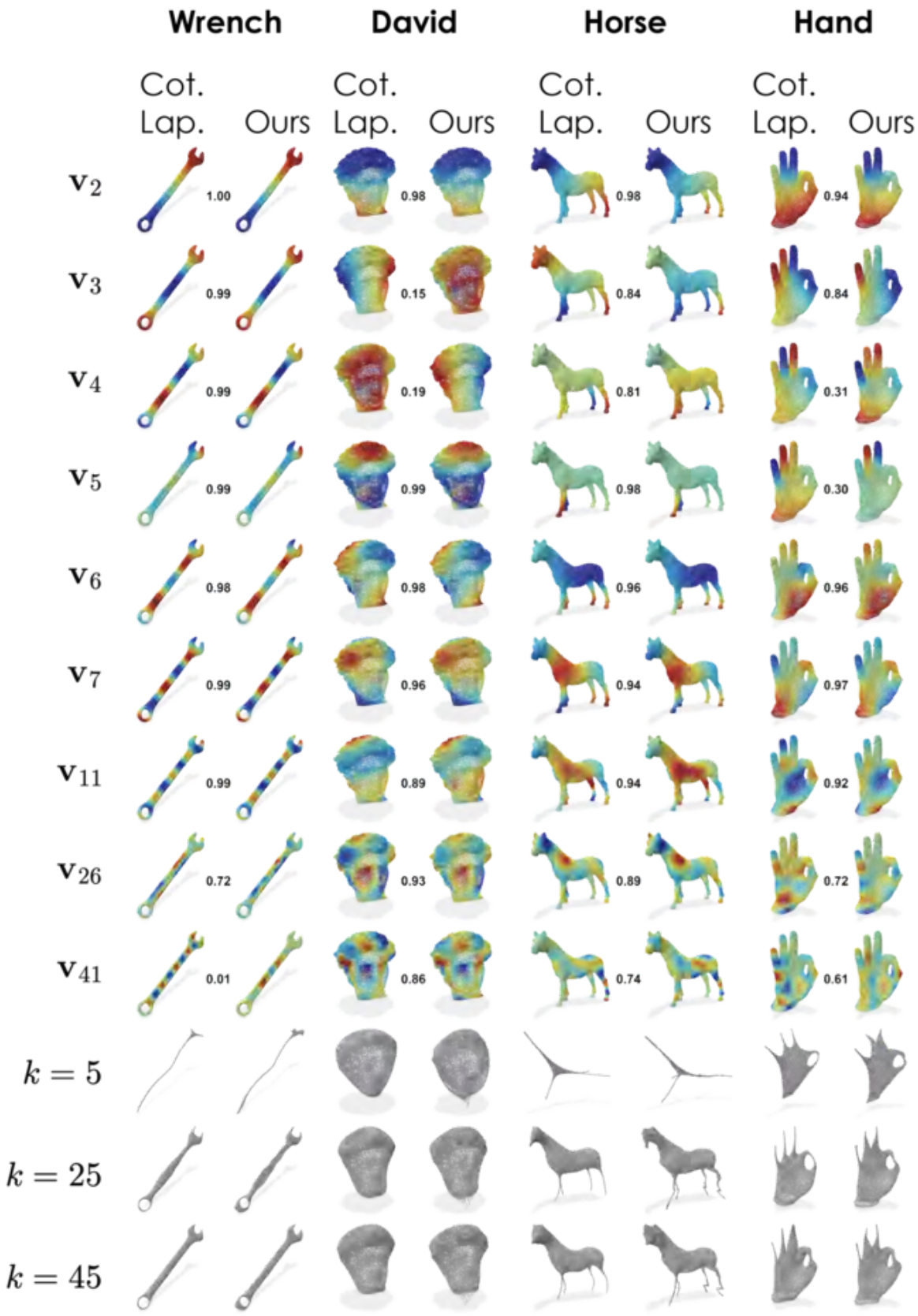


Figure 14. Unnormalized spectral basis (top) and xyz reconstruction from k basis vectors (bottom), using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

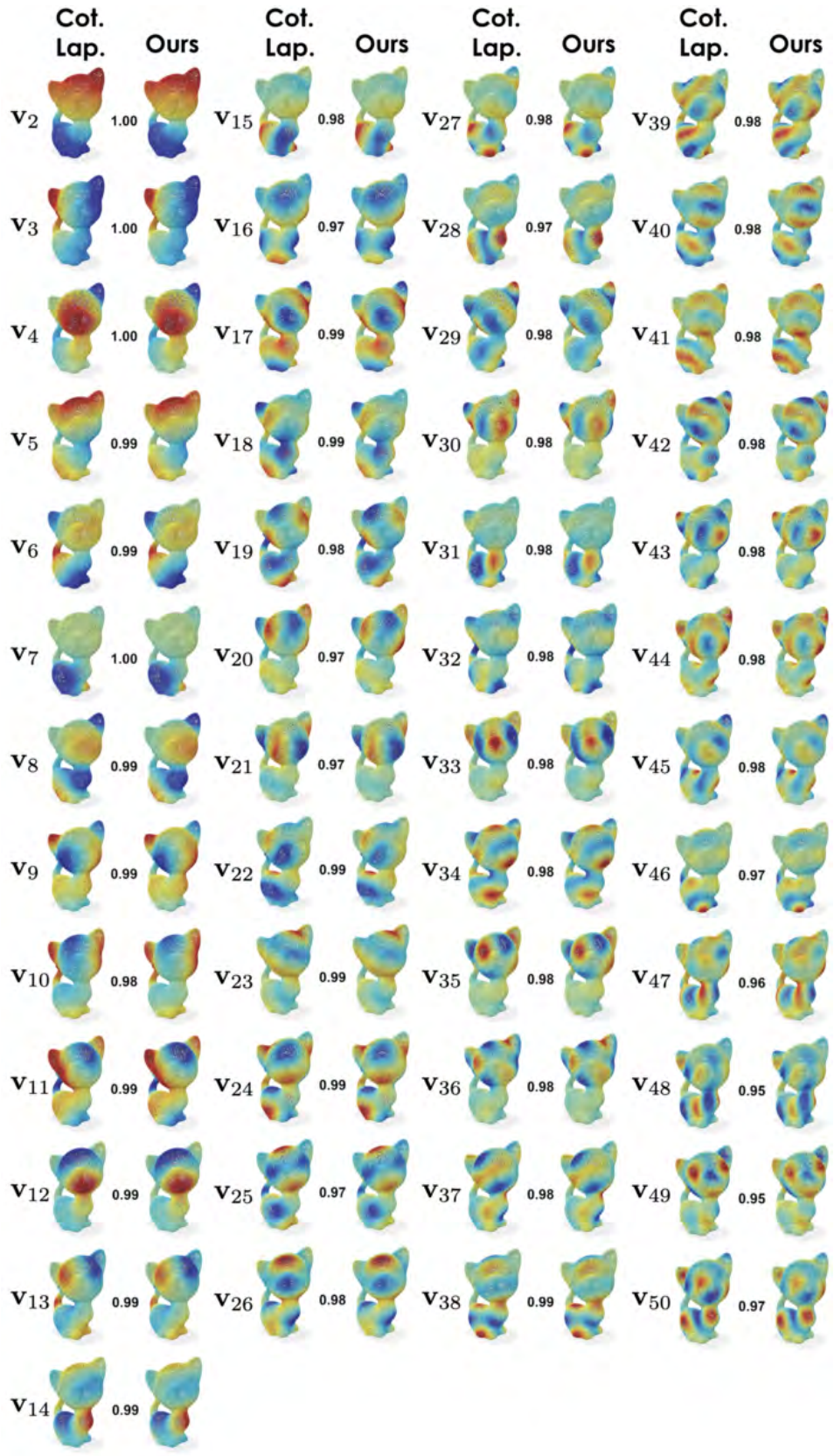


Figure 15. Unnormalized spectral basis using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

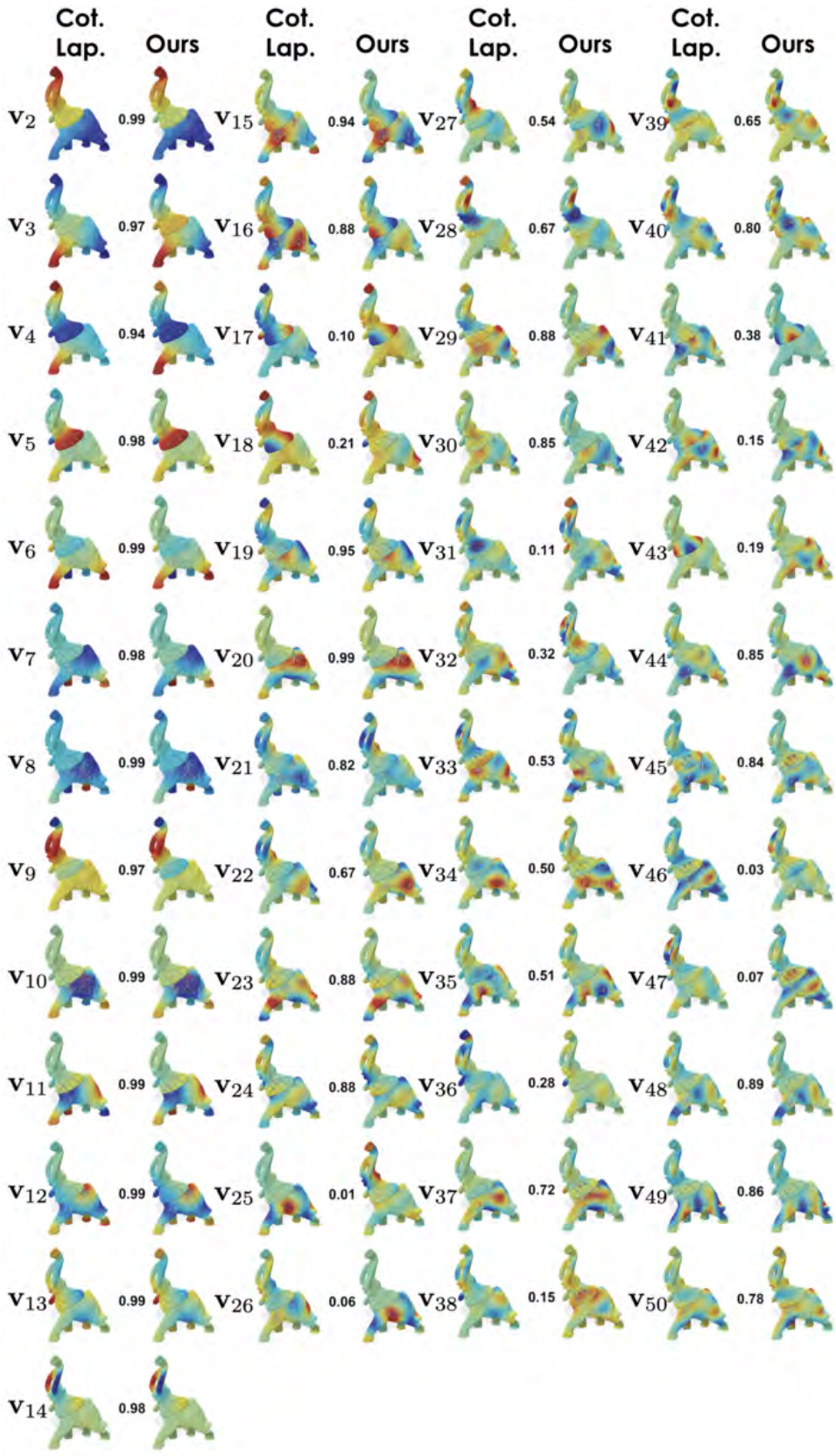


Figure 16. Unnormalized spectral basis using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

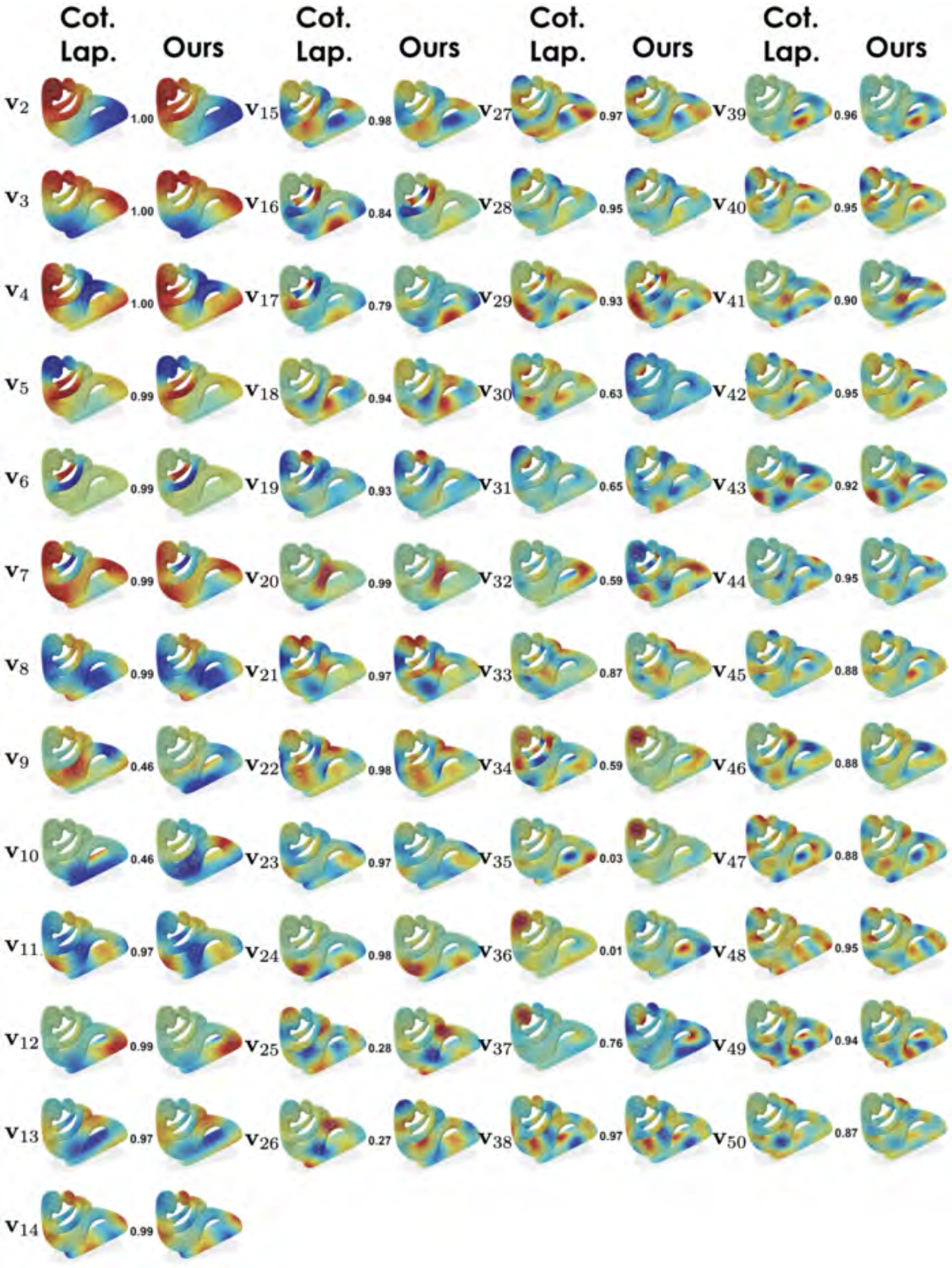


Figure 17. Unnormalized spectral basis using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

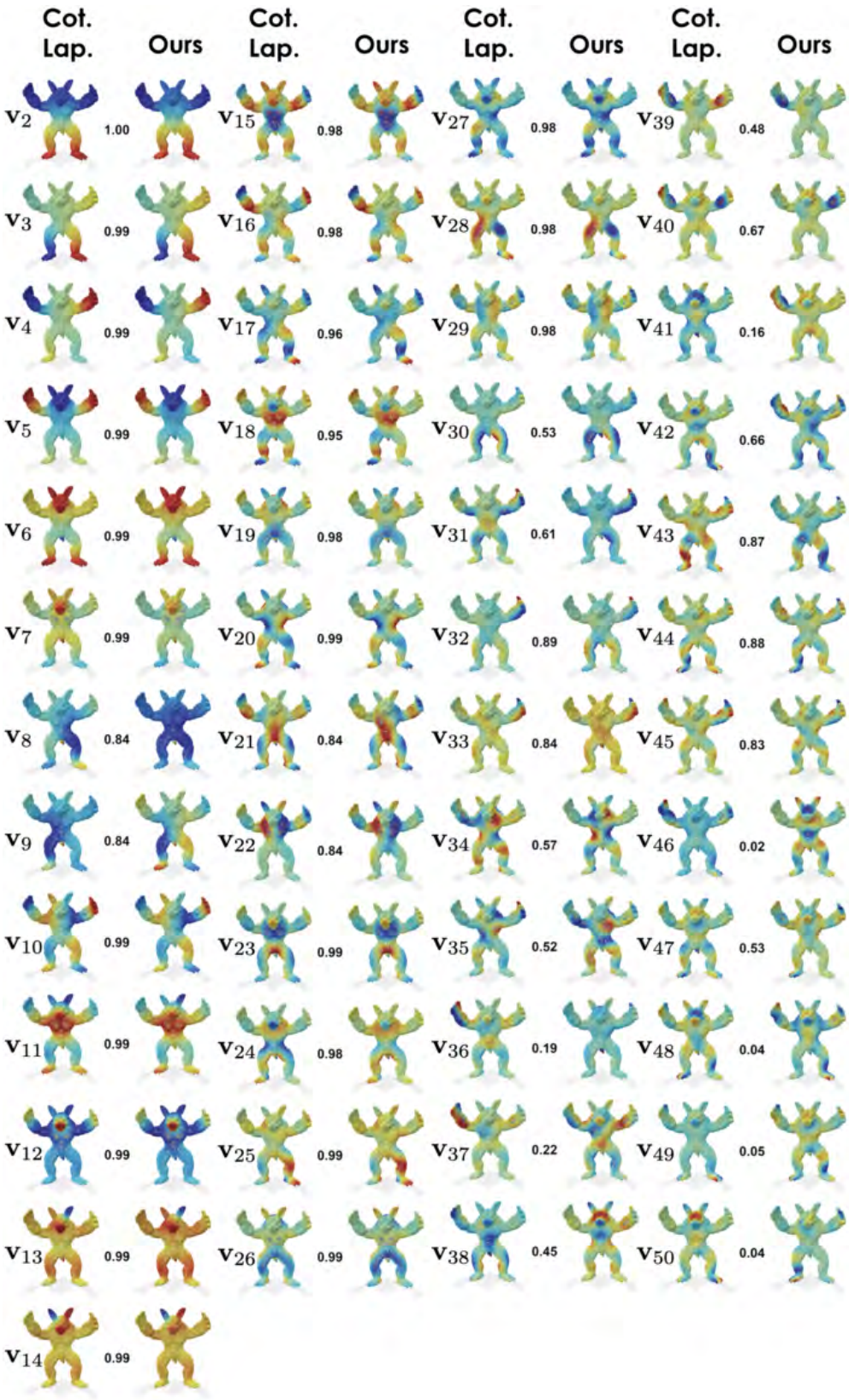


Figure 18. Unnormalized spectral basis using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

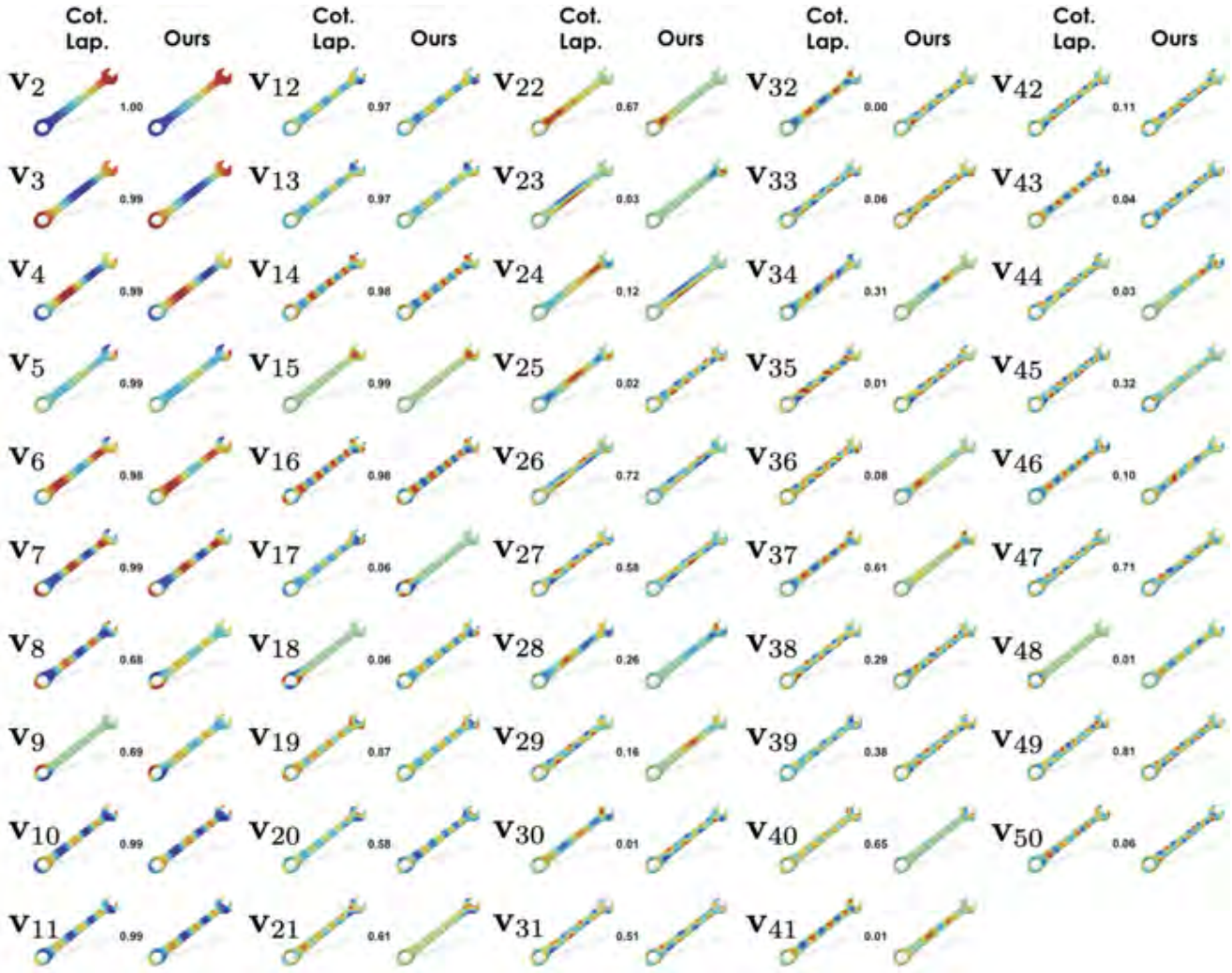


Figure 19. Unnormalized spectral basis using either the oracle cotangent Laplacian or our method (overfitting setting). Scalars are cosine similarities between basis vectors.

Figure 20. Estimated mass metric M from $\mathbf{q}_1$ (overfitting setting).

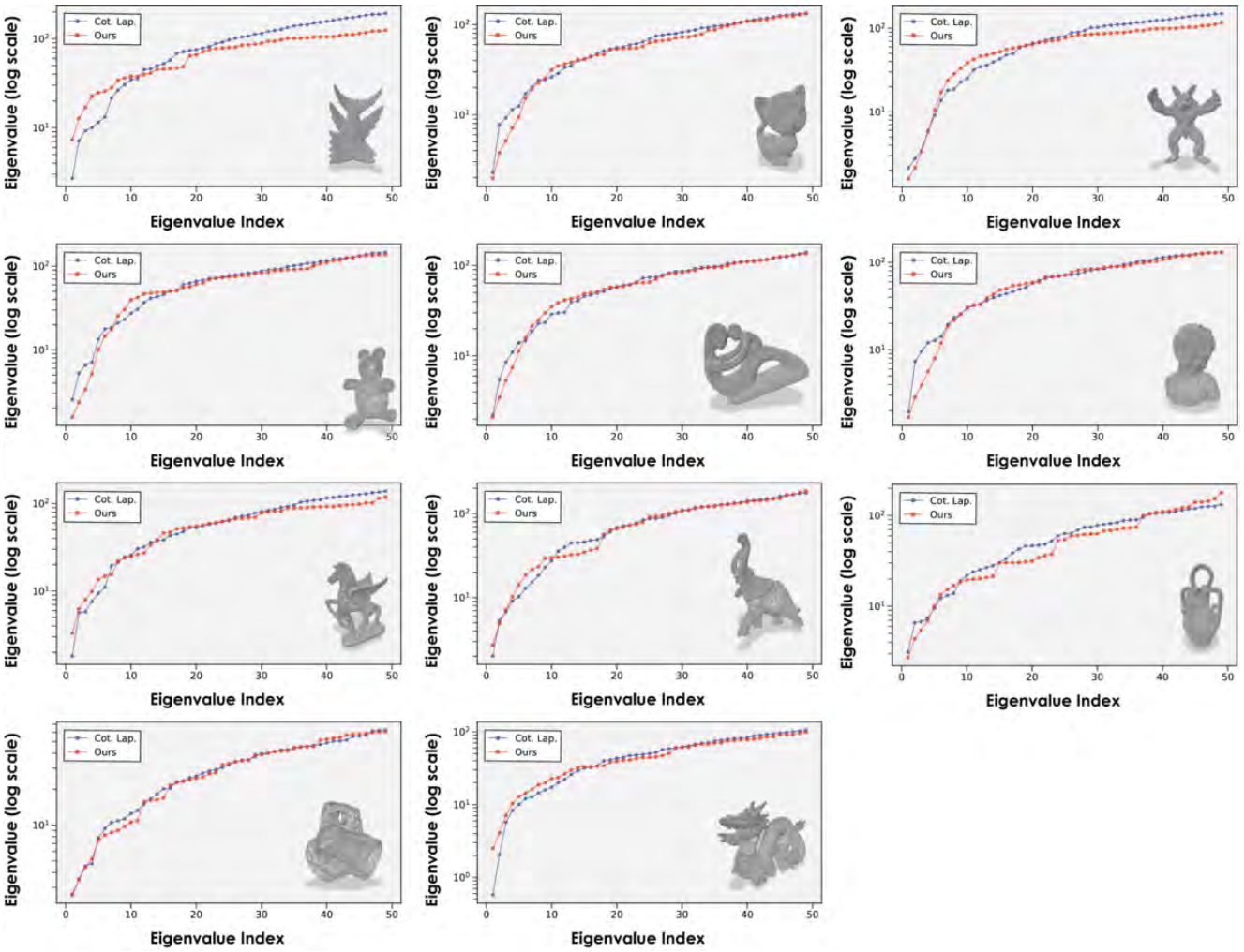


Figure 21. Eigenvalues of the oracle cotangent Laplacian and our estimated ones (overfitting setting, fine-tuned hyperparameter configuration for generating probe functions per shape).

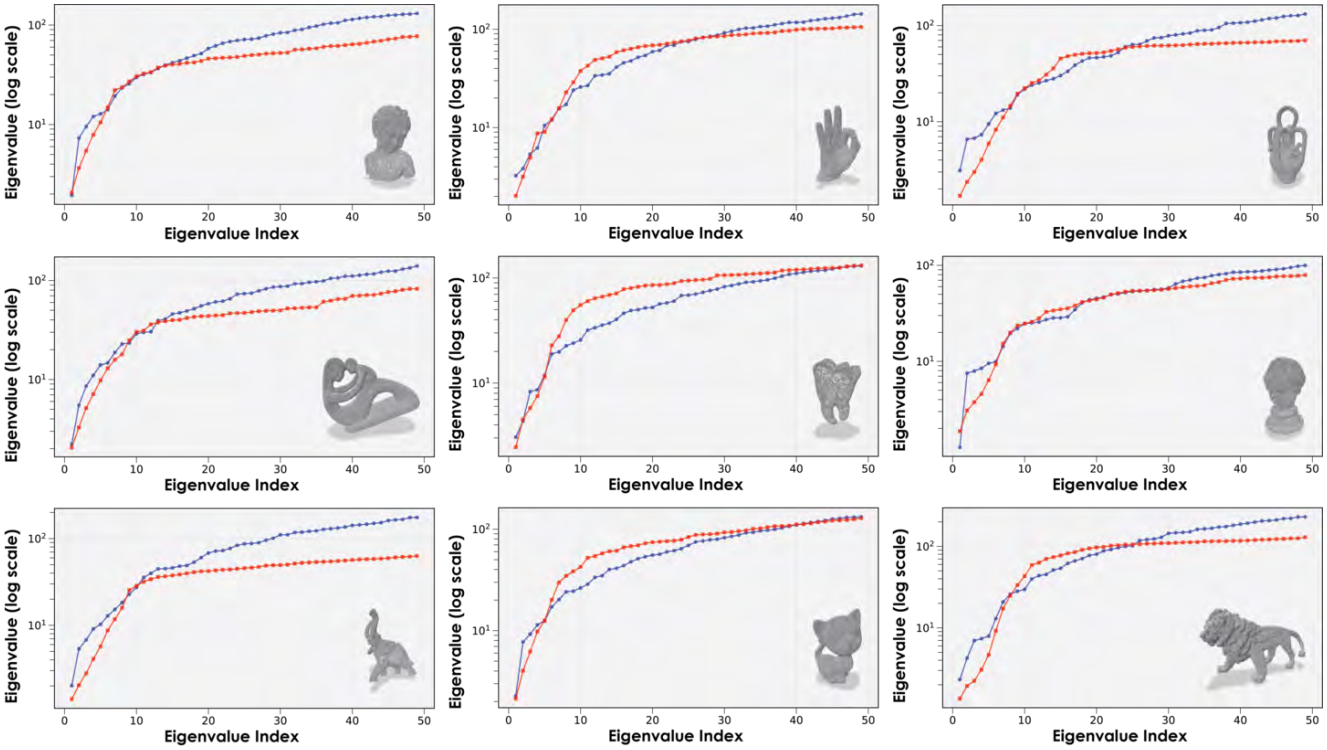


Figure 22. Eigenvalues of the oracle cotangent Laplacian and our estimated ones (overfitting setting, fixed hyperparameter configuration for generating probe functions across all shapes).

Table 5. Average cosine similarity between predicted and oracle eigenfunctions at different truncation levels k on each evaluation dataset (generalization setting).

Dataset	**#Shapes**	$k \leq 5$	$k \leq 10$	$k \leq 15$	$k \leq 20$	$k \leq 25$	$k \leq 30$	$k \leq 35$	$k \leq 40$	$k \leq 45$	$k \leq 50$
MPZ14	105	0.735	0.612	0.526	0.473	0.429	0.393	0.365	0.341	0.320	0.301
SHREC'07	380	0.729	0.610	0.535	0.480	0.438	0.403	0.374	0.349	0.327	0.308
SHREC'14	700	0.793	0.666	0.585	0.532	0.478	0.438	0.404	0.377	0.352	0.329
SHREC'15	1200	0.693	0.570	0.511	0.458	0.417	0.384	0.356	0.331	0.310	0.291
SHREC'19	50	0.750	0.633	0.556	0.495	0.448	0.409	0.381	0.357	0.334	0.315
SHREC'20-GR	220	0.670	0.514	0.435	0.385	0.348	0.321	0.296	0.277	0.261	0.247
SHREC'20-NI	14	0.598	0.536	0.495	0.446	0.404	0.375	0.348	0.326	0.307	0.291
DefTransfer	278	0.621	0.500	0.425	0.386	0.352	0.326	0.303	0.285	0.269	0.254
TopKids	26	0.788	0.656	0.551	0.486	0.438	0.402	0.374	0.352	0.331	0.312

Table 6. Average manifold learning results on STL10 with DINOv2 embeddings. The mean and standard deviation over 32 runs is provided. The best and second best results are in bold and underlined respectively.

STL10 - DINOv2								
Method	k	NMI	ARI	Comp.	AMI	Homo.	V-Meas.	FMI
Opt. App. Eigenmaps (Ours)	2	$\underline{0.790 \pm 0.083}$	$\mathbf{0.712 \pm 0.144}$	$\mathbf{0.752 \pm 0.094}$	$\underline{0.787 \pm 0.084}$	0.834 ± 0.080	$\underline{0.790 \pm 0.083}$	$\mathbf{0.775 \pm 0.110}$
Laplacian Eigenmaps	2	0.594 ± 0.075	0.382 ± 0.096	0.568 ± 0.058	0.589 ± 0.076	0.627 ± 0.111	0.594 ± 0.075	0.510 ± 0.095
PCA	2	0.668 ± 0.046	0.475 ± 0.073	0.611 ± 0.041	0.664 ± 0.046	0.740 ± 0.075	0.668 ± 0.046	0.579 ± 0.076
UMAP	2	$\mathbf{0.808 \pm 0.058}$	$\underline{0.633 \pm 0.119}$	$\underline{0.741 \pm 0.082}$	$\mathbf{0.806 \pm 0.059}$	$\mathbf{0.894 \pm 0.038}$	$\mathbf{0.808 \pm 0.058}$	$\underline{0.717 \pm 0.081}$
t-SNE	2	0.789 ± 0.058	0.595 ± 0.106	0.718 ± 0.080	0.786 ± 0.058	$\underline{0.879 \pm 0.033}$	0.789 ± 0.058	0.686 ± 0.070
Isomap	2	0.684 ± 0.052	0.520 ± 0.092	0.632 ± 0.062	0.681 ± 0.053	0.751 ± 0.066	0.684 ± 0.052	0.618 ± 0.074
Opt. App. Eigenmaps (Ours)	5	$\mathbf{0.859 \pm 0.061}$	$\mathbf{0.812 \pm 0.120}$	$\mathbf{0.823 \pm 0.081}$	$\mathbf{0.858 \pm 0.061}$	$\mathbf{0.902 \pm 0.047}$	$\mathbf{0.859 \pm 0.061}$	$\mathbf{0.856 \pm 0.088}$
Laplacian Eigenmaps	5	0.815 ± 0.054	$\underline{0.734 \pm 0.130}$	$\underline{0.793 \pm 0.063}$	0.813 ± 0.055	0.842 ± 0.065	0.815 ± 0.054	$\underline{0.792 \pm 0.104}$
PCA	5	0.746 ± 0.053	0.565 ± 0.092	0.682 ± 0.073	0.743 ± 0.053	0.828 ± 0.043	0.746 ± 0.053	0.658 ± 0.061
UMAP	5	$\underline{0.821 \pm 0.051}$	0.666 ± 0.109	0.757 ± 0.076	$\underline{0.819 \pm 0.051}$	$\underline{0.901 \pm 0.031}$	$\underline{0.821 \pm 0.051}$	0.743 ± 0.074
t-SNE	5	0.796 ± 0.060	0.616 ± 0.112	0.727 ± 0.083	0.793 ± 0.060	0.884 ± 0.037	0.796 ± 0.060	0.704 ± 0.075
Isomap	5	0.772 ± 0.047	0.626 ± 0.104	0.711 ± 0.071	0.769 ± 0.048	0.849 ± 0.037	0.772 ± 0.047	0.709 ± 0.069
Opt. App. Eigenmaps (Ours)	10	$\mathbf{0.892 \pm 0.047}$	$\mathbf{0.863 \pm 0.098}$	$\mathbf{0.862 \pm 0.071}$	$\mathbf{0.891 \pm 0.048}$	$\mathbf{0.927 \pm 0.029}$	$\mathbf{0.892 \pm 0.047}$	$\mathbf{0.896 \pm 0.071}$
Laplacian Eigenmaps	10	$\underline{0.828 \pm 0.077}$	$\underline{0.768 \pm 0.124}$	$\underline{0.835 \pm 0.068}$	$\underline{0.826 \pm 0.077}$	0.827 ± 0.101	$\underline{0.828 \pm 0.077}$	$\underline{0.829 \pm 0.073}$
PCA	10	0.788 ± 0.045	0.620 ± 0.097	0.721 ± 0.068	0.785 ± 0.046	0.873 ± 0.036	0.788 ± 0.045	0.706 ± 0.065
UMAP	10	0.821 ± 0.051	0.665 ± 0.109	0.758 ± 0.076	0.819 ± 0.051	$\underline{0.901 \pm 0.031}$	0.821 ± 0.051	0.742 ± 0.074
t-SNE	10	0.810 ± 0.051	0.643 ± 0.108	0.743 ± 0.077	0.808 ± 0.051	0.896 ± 0.030	0.810 ± 0.051	0.726 ± 0.071
Isomap	10	0.794 ± 0.050	0.656 ± 0.101	0.733 ± 0.072	0.791 ± 0.050	0.870 ± 0.037	0.794 ± 0.050	0.733 ± 0.068

Table 7. Average manifold learning results on STL10 with CLIP embeddings. The mean and standard deviation over 32 runs is provided. The best and second best results are in bold and underlined respectively.

STL10 - CLIP								
Method	k	NMI	ARI	Comp.	AMI	Homo.	V-Meas.	FMI
Opt. App. Eigenmaps (Ours)	2	0.691 ± 0.071	0.525 ± 0.108	0.653 ± 0.071	0.687 ± 0.072	0.736 ± 0.085	0.691 ± 0.071	0.622 ± 0.093
Laplacian Eigenmaps	2	0.585 ± 0.071	0.365 ± 0.109	0.558 ± 0.056	0.580 ± 0.073	0.621 ± 0.112	0.585 ± 0.071	0.493 ± 0.110
PCA	2	0.673 ± 0.041	0.515 ± 0.071	0.615 ± 0.043	0.669 ± 0.042	0.746 ± 0.063	0.673 ± 0.041	0.613 ± 0.069
UMAP	2	$\mathbf{0.835 \pm 0.050}$	$\mathbf{0.676 \pm 0.111}$	$\mathbf{0.770 \pm 0.078}$	$\mathbf{0.833 \pm 0.051}$	$\mathbf{0.915 \pm 0.028}$	$\mathbf{0.835 \pm 0.050}$	$\mathbf{0.752 \pm 0.075}$
t-SNE	2	$\underline{0.801 \pm 0.056}$	$\underline{0.614 \pm 0.110}$	$\underline{0.730 \pm 0.080}$	$\underline{0.798 \pm 0.057}$	$\underline{0.892 \pm 0.032}$	$\underline{0.801 \pm 0.056}$	$\underline{0.703 \pm 0.072}$
Isomap	2	0.718 ± 0.053	0.590 ± 0.113	0.667 ± 0.065	0.715 ± 0.053	0.781 ± 0.060	0.718 ± 0.053	0.675 ± 0.095
Opt. App. Eigenmaps (Ours)	5	$\underline{0.820 \pm 0.060}$	$\mathbf{0.741 \pm 0.120}$	$\mathbf{0.795 \pm 0.072}$	$\underline{0.818 \pm 0.060}$	0.850 ± 0.063	$\underline{0.820 \pm 0.060}$	$\mathbf{0.797 \pm 0.094}$
Laplacian Eigenmaps	5	0.796 ± 0.042	$\underline{0.716 \pm 0.107}$	0.774 ± 0.063	0.794 ± 0.043	0.823 ± 0.045	0.796 ± 0.042	$\underline{0.775 \pm 0.095}$
PCA	5	0.760 ± 0.043	0.593 ± 0.091	0.694 ± 0.061	0.757 ± 0.044	0.844 ± 0.041	0.760 ± 0.043	0.682 ± 0.066
UMAP	5	$\mathbf{0.838 \pm 0.049}$	0.694 ± 0.103	$\underline{0.777 \pm 0.074}$	$\mathbf{0.837 \pm 0.050}$	$\mathbf{0.914 \pm 0.028}$	$\mathbf{0.838 \pm 0.049}$	0.765 ± 0.070
t-SNE	5	0.803 ± 0.064	0.632 ± 0.118	0.735 ± 0.088	0.801 ± 0.064	$\underline{0.890 \pm 0.036}$	0.803 ± 0.064	0.717 ± 0.080
Isomap	5	0.796 ± 0.041	0.672 ± 0.102	0.741 ± 0.063	0.794 ± 0.042	0.865 ± 0.032	0.796 ± 0.041	0.745 ± 0.077
Opt. App. Eigenmaps (Ours)	10	$\mathbf{0.863 \pm 0.047}$	$\mathbf{0.818 \pm 0.086}$	$\underline{0.845 \pm 0.063}$	$\mathbf{0.861 \pm 0.047}$	0.884 ± 0.047	$\mathbf{0.863 \pm 0.047}$	$\mathbf{0.858 \pm 0.067}$
Laplacian Eigenmaps	10	0.796 ± 0.131	$\underline{0.720 \pm 0.166}$	$\mathbf{0.850 \pm 0.055}$	0.794 ± 0.133	0.770 ± 0.161	0.796 ± 0.131	$\underline{0.803 \pm 0.086}$
PCA	10	0.788 ± 0.051	0.628 ± 0.103	0.723 ± 0.072	0.785 ± 0.051	0.870 ± 0.035	0.788 ± 0.051	0.711 ± 0.073
UMAP	10	$\underline{0.833 \pm 0.049}$	0.685 ± 0.103	0.772 ± 0.074	$\underline{0.832 \pm 0.049}$	$\mathbf{0.910 \pm 0.029}$	$\underline{0.833 \pm 0.049}$	0.758 ± 0.070
t-SNE	10	0.811 ± 0.066	0.650 ± 0.118	0.745 ± 0.090	0.809 ± 0.067	$\underline{0.894 \pm 0.038}$	0.811 ± 0.066	0.731 ± 0.080
Isomap	10	0.814 ± 0.047	0.697 ± 0.111	0.759 ± 0.069	0.812 ± 0.048	0.881 ± 0.031	0.814 ± 0.047	0.765 ± 0.082

Table 8. Average manifold learning results on Imagenette with DINOv2 embeddings. The mean and standard deviation over 32 runs is provided. The best and second best results are in bold and underlined respectively.

Imagenette - DINOv2								
Method	k	NMI	ARI	Comp.	AMI	Homo.	V-Meas.	FMI
Opt. App. Eigenmaps (Ours)	2	0.684 ± 0.061	0.602 ± 0.114	0.655 ± 0.070	0.680 ± 0.062	0.716 ± 0.053	0.684 ± 0.061	0.667 ± 0.098
Laplacian Eigenmaps	2	0.642 ± 0.062	0.456 ± 0.102	0.627 ± 0.048	0.638 ± 0.063	0.659 ± 0.082	0.642 ± 0.062	0.550 ± 0.091
PCA	2	0.645 ± 0.075	0.486 ± 0.126	0.617 ± 0.062	0.641 ± 0.077	0.677 ± 0.092	0.645 ± 0.075	0.568 ± 0.113
UMAP	2	$\mathbf{0.875 \pm 0.035}$	$\mathbf{0.802 \pm 0.076}$	$\mathbf{0.836 \pm 0.053}$	$\mathbf{0.873 \pm 0.036}$	$\mathbf{0.918 \pm 0.023}$	$\mathbf{0.875 \pm 0.035}$	$\mathbf{0.839 \pm 0.060}$
t-SNE	2	$\underline{0.829 \pm 0.036}$	$\underline{0.716 \pm 0.072}$	$\underline{0.782 \pm 0.045}$	$\underline{0.828 \pm 0.036}$	$\underline{0.884 \pm 0.036}$	$\underline{0.829 \pm 0.036}$	$\underline{0.767 \pm 0.058}$
Isomap	2	0.679 ± 0.045	0.599 ± 0.090	0.646 ± 0.041	0.676 ± 0.046	0.717 ± 0.058	0.679 ± 0.045	0.665 ± 0.080
Opt. App. Eigenmaps (Ours)	5	$\underline{0.842 \pm 0.043}$	$\mathbf{0.844 \pm 0.051}$	$\underline{0.818 \pm 0.058}$	$\underline{0.840 \pm 0.044}$	0.868 ± 0.032	$\underline{0.842 \pm 0.043}$	$\mathbf{0.872 \pm 0.041}$
Laplacian Eigenmaps	5	0.800 ± 0.069	0.703 ± 0.137	0.787 ± 0.071	0.798 ± 0.070	0.817 ± 0.081	0.800 ± 0.069	0.754 ± 0.117
PCA	5	0.759 ± 0.048	0.629 ± 0.080	0.718 ± 0.049	0.756 ± 0.048	0.806 ± 0.056	0.759 ± 0.048	0.691 ± 0.070
UMAP	5	$\mathbf{0.881 \pm 0.038}$	$\underline{0.816 \pm 0.086}$	$\mathbf{0.845 \pm 0.058}$	$\mathbf{0.880 \pm 0.039}$	$\mathbf{0.922 \pm 0.021}$	$\mathbf{0.881 \pm 0.038}$	$\underline{0.850 \pm 0.068}$
t-SNE	5	0.825 ± 0.029	0.725 ± 0.059	0.779 ± 0.040	0.823 ± 0.029	$\underline{0.879 \pm 0.030}$	0.825 ± 0.029	0.775 ± 0.046
Isomap	5	0.836 ± 0.039	0.790 ± 0.077	0.802 ± 0.049	0.834 ± 0.040	0.875 ± 0.045	0.836 ± 0.039	0.827 ± 0.065
Opt. App. Eigenmaps (Ours)	10	0.858 ± 0.046	$\mathbf{0.856 \pm 0.055}$	$\underline{0.832 \pm 0.060}$	0.857 ± 0.046	0.888 ± 0.037	0.858 ± 0.046	$\mathbf{0.881 \pm 0.044}$
Laplacian Eigenmaps	10	0.752 ± 0.092	0.577 ± 0.145	0.820 ± 0.061	0.749 ± 0.092	0.706 ± 0.122	0.752 ± 0.092	0.685 ± 0.091
PCA	10	0.795 ± 0.047	0.676 ± 0.080	0.754 ± 0.052	0.793 ± 0.047	0.841 ± 0.052	0.795 ± 0.047	0.732 ± 0.067
UMAP	10	$\mathbf{0.882 \pm 0.038}$	$\underline{0.818 \pm 0.086}$	$\mathbf{0.846 \pm 0.058}$	$\mathbf{0.881 \pm 0.039}$	$\mathbf{0.923 \pm 0.021}$	$\mathbf{0.882 \pm 0.038}$	0.851 ± 0.068
t-SNE	10	0.820 ± 0.036	0.721 ± 0.068	0.776 ± 0.047	0.818 ± 0.037	0.871 ± 0.034	0.820 ± 0.036	0.771 ± 0.055
Isomap	10	$\underline{0.862 \pm 0.043}$	0.818 ± 0.093	0.828 ± 0.063	$\underline{0.860 \pm 0.043}$	$\underline{0.901 \pm 0.030}$	$\underline{0.862 \pm 0.043}$	$\underline{0.851 \pm 0.073}$

Table 9. Average manifold learning results on Imagenette with CLIP embeddings. The mean and standard deviation over 32 runs is provided. The best and second best results are in bold and underlined respectively.

Imagenette - CLIP								
Method	k	NMI	ARI	Comp.	AMI	Homo.	V-Meas.	FMI
Opt. App. Eigenmaps (Ours)	2	0.746 ± 0.055	0.680 ± 0.083	0.712 ± 0.061	0.743 ± 0.056	0.784 ± 0.056	0.746 ± 0.055	0.733 ± 0.069
Laplacian Eigenmaps	2	0.519 ± 0.109	0.281 ± 0.130	0.532 ± 0.083	0.513 ± 0.112	0.515 ± 0.140	0.519 ± 0.109	0.421 ± 0.104
PCA	2	0.682 ± 0.043	0.551 ± 0.073	0.642 ± 0.034	0.678 ± 0.044	0.728 ± 0.060	0.682 ± 0.043	0.623 ± 0.071
UMAP	2	$\mathbf{0.895 \pm 0.042}$	$\mathbf{0.818 \pm 0.095}$	$\mathbf{0.855 \pm 0.065}$	$\mathbf{0.894 \pm 0.042}$	$\mathbf{0.941 \pm 0.017}$	$\mathbf{0.895 \pm 0.042}$	$\mathbf{0.853 \pm 0.074}$
t-SNE	2	$\underline{0.876 \pm 0.041}$	$\underline{0.782 \pm 0.091}$	$\underline{0.831 \pm 0.063}$	$\underline{0.875 \pm 0.042}$	$\underline{0.929 \pm 0.022}$	$\underline{0.876 \pm 0.041}$	$\underline{0.823 \pm 0.071}$
Isomap	2	0.755 ± 0.040	0.683 ± 0.074	0.721 ± 0.039	0.752 ± 0.040	0.794 ± 0.054	0.755 ± 0.040	0.735 ± 0.066
Opt. App. Eigenmaps (Ours)	5	$\underline{0.882 \pm 0.034}$	$\mathbf{0.874 \pm 0.055}$	$\underline{0.858 \pm 0.046}$	$\underline{0.880 \pm 0.034}$	0.908 ± 0.026	$\underline{0.882 \pm 0.034}$	$\mathbf{0.896 \pm 0.044}$
Laplacian Eigenmaps	5	0.820 ± 0.058	0.707 ± 0.141	0.809 ± 0.053	0.818 ± 0.058	0.833 ± 0.074	0.820 ± 0.058	0.757 ± 0.119
PCA	5	0.818 ± 0.040	0.746 ± 0.080	0.774 ± 0.050	0.816 ± 0.040	0.869 ± 0.039	0.818 ± 0.040	0.792 ± 0.064
UMAP	5	$\mathbf{0.898 \pm 0.043}$	0.830 ± 0.094	$\mathbf{0.859 \pm 0.066}$	$\mathbf{0.897 \pm 0.043}$	$\mathbf{0.942 \pm 0.020}$	$\mathbf{0.898 \pm 0.043}$	0.862 ± 0.073
t-SNE	5	0.875 ± 0.042	0.795 ± 0.093	0.832 ± 0.065	0.874 ± 0.043	$\underline{0.925 \pm 0.022}$	0.875 ± 0.042	0.834 ± 0.072
Isomap	5	0.876 ± 0.030	$\underline{0.844 \pm 0.064}$	0.842 ± 0.046	0.874 ± 0.030	0.914 ± 0.027	0.876 ± 0.030	$\underline{0.872 \pm 0.052}$
Opt. App. Eigenmaps (Ours)	10	$\mathbf{0.915 \pm 0.028}$	$\mathbf{0.917 \pm 0.045}$	$\underline{0.895 \pm 0.042}$	$\mathbf{0.914 \pm 0.029}$	$\underline{0.937 \pm 0.017}$	$\mathbf{0.915 \pm 0.028}$	$\mathbf{0.932 \pm 0.036}$
Laplacian Eigenmaps	10	0.819 ± 0.167	0.707 ± 0.193	$\mathbf{0.899 \pm 0.141}$	0.817 ± 0.168	0.761 ± 0.174	0.819 ± 0.167	0.790 ± 0.108
PCA	10	0.855 ± 0.034	0.780 ± 0.077	0.811 ± 0.051	0.854 ± 0.034	0.907 ± 0.025	0.855 ± 0.034	0.821 ± 0.060
UMAP	10	$\underline{0.899 \pm 0.041}$	0.830 ± 0.091	0.860 ± 0.064	$\underline{0.898 \pm 0.042}$	$\mathbf{0.943 \pm 0.018}$	$\underline{0.899 \pm 0.041}$	0.863 ± 0.071
t-SNE	10	0.876 ± 0.039	0.793 ± 0.084	0.832 ± 0.060	0.874 ± 0.040	0.926 ± 0.022	0.876 ± 0.039	0.832 ± 0.064
Isomap	10	0.895 ± 0.038	$\underline{0.852 \pm 0.080}$	0.860 ± 0.059	0.894 ± 0.038	0.934 ± 0.020	0.895 ± 0.038	$\underline{0.880 \pm 0.063}$

Table 10. Average manifold learning results on CALTECH256 with DINOv2 embeddings. The mean and standard deviation over 32 runs is provided. The best and second best results are in bold and underlined respectively.

CALTECH256 - DINOv2								
Method	k	NMI	ARI	Comp.	AMI	Homo.	V-Meas.	FMI
Opt. App. Eigenmaps (Ours)	2	0.611 ± 0.027	0.078 ± 0.026	0.594 ± 0.030	0.196 ± 0.053	0.629 ± 0.024	0.611 ± 0.027	0.092 ± 0.031
Laplacian Eigenmaps	2	0.650 ± 0.036	0.121 ± 0.037	0.645 ± 0.038	0.311 ± 0.061	0.655 ± 0.036	0.650 ± 0.036	0.134 ± 0.036
PCA	2	0.603 ± 0.029	0.063 ± 0.020	0.583 ± 0.032	0.165 ± 0.050	0.626 ± 0.027	0.603 ± 0.029	0.077 ± 0.026
UMAP	2	$\underline{0.834 \pm 0.014}$	$\mathbf{0.448 \pm 0.084}$	$\mathbf{0.810 \pm 0.020}$	$\underline{0.654 \pm 0.048}$	$\underline{0.860 \pm 0.013}$	$\underline{0.834 \pm 0.014}$	$\mathbf{0.476 \pm 0.073}$
t-SNE	2	$\mathbf{0.840 \pm 0.015}$	$\underline{0.419 \pm 0.071}$	$\underline{0.809 \pm 0.021}$	$\mathbf{0.660 \pm 0.036}$	$\mathbf{0.873 \pm 0.011}$	$\mathbf{0.840 \pm 0.015}$	$\underline{0.457 \pm 0.059}$
Isomap	2	0.655 ± 0.021	0.138 ± 0.045	0.633 ± 0.024	0.273 ± 0.063	0.678 ± 0.019	0.655 ± 0.021	0.155 ± 0.051
Opt. App. Eigenmaps (Ours)	5	0.714 ± 0.020	0.246 ± 0.062	0.696 ± 0.024	0.411 ± 0.058	0.733 ± 0.019	0.714 ± 0.020	0.263 ± 0.067
Laplacian Eigenmaps	5	0.724 ± 0.024	0.209 ± 0.044	0.712 ± 0.026	0.445 ± 0.053	0.737 ± 0.026	0.724 ± 0.024	0.223 ± 0.042
PCA	5	0.696 ± 0.019	0.184 ± 0.039	0.671 ± 0.023	0.355 ± 0.054	0.722 ± 0.015	0.696 ± 0.019	0.206 ± 0.043
UMAP	5	$\mathbf{0.843 \pm 0.013}$	$\mathbf{0.465 \pm 0.081}$	$\mathbf{0.820 \pm 0.020}$	$\mathbf{0.675 \pm 0.039}$	$\mathbf{0.868 \pm 0.011}$	$\mathbf{0.843 \pm 0.013}$	$\mathbf{0.493 \pm 0.068}$
t-SNE	5	$\underline{0.832 \pm 0.015}$	$\underline{0.394 \pm 0.066}$	$\underline{0.800 \pm 0.020}$	$\underline{0.642 \pm 0.039}$	$\underline{0.866 \pm 0.011}$	$\underline{0.832 \pm 0.015}$	$\underline{0.432 \pm 0.056}$
Isomap	5	0.754 ± 0.017	0.319 ± 0.067	0.732 ± 0.021	0.486 ± 0.056	0.778 ± 0.015	0.754 ± 0.017	0.341 ± 0.067
Opt. App. Eigenmaps (Ours)	10	0.759 ± 0.016	0.322 ± 0.060	0.743 ± 0.021	0.508 ± 0.047	0.775 ± 0.014	0.759 ± 0.016	0.339 ± 0.062
Laplacian Eigenmaps	10	0.758 ± 0.016	0.257 ± 0.042	0.743 ± 0.021	0.508 ± 0.045	0.774 ± 0.015	0.758 ± 0.016	0.272 ± 0.039
PCA	10	0.747 ± 0.015	0.267 ± 0.049	0.721 ± 0.020	0.466 ± 0.052	0.776 ± 0.013	0.747 ± 0.015	0.293 ± 0.050
UMAP	10	$\mathbf{0.843 \pm 0.013}$	$\mathbf{0.462 \pm 0.083}$	$\mathbf{0.819 \pm 0.021}$	$\mathbf{0.673 \pm 0.040}$	$\mathbf{0.868 \pm 0.011}$	$\mathbf{0.843 \pm 0.013}$	$\mathbf{0.490 \pm 0.070}$
t-SNE	10	$\underline{0.823 \pm 0.015}$	0.379 ± 0.066	$\underline{0.792 \pm 0.021}$	$\underline{0.623 \pm 0.039}$	$\underline{0.857 \pm 0.011}$	$\underline{0.823 \pm 0.015}$	0.416 ± 0.055
Isomap	10	0.799 ± 0.015	$\underline{0.409 \pm 0.072}$	0.777 ± 0.019	0.582 ± 0.050	0.823 ± 0.012	0.799 ± 0.015	$\underline{0.433 \pm 0.068}$

Table 11. Average manifold learning results on CALTECH256 with CLIP embeddings. The mean and standard deviation over 32 runs is provided. The best and second best results are in bold and underlined respectively.

CALTECH256 - CLIP								
Method	k	NMI	ARI	Comp.	AMI	Homo.	V-Meas.	FMI
Opt. App. Eigenmaps (Ours)	2	0.596 ± 0.031	0.069 ± 0.030	0.578 ± 0.033	0.159 ± 0.050	0.615 ± 0.029	0.596 ± 0.031	0.083 ± 0.036
Laplacian Eigenmaps	2	0.600 ± 0.055	0.075 ± 0.032	0.599 ± 0.055	0.225 ± 0.076	0.602 ± 0.057	0.600 ± 0.055	0.088 ± 0.032
PCA	2	0.599 ± 0.031	0.062 ± 0.022	0.577 ± 0.034	0.153 ± 0.046	0.622 ± 0.028	0.599 ± 0.031	0.076 ± 0.028
UMAP	2	$\underline{0.844 \pm 0.015}$	$\mathbf{0.472 \pm 0.094}$	$\underline{0.819 \pm 0.021}$	$\underline{0.673 \pm 0.051}$	$\underline{0.869 \pm 0.013}$	$\underline{0.844 \pm 0.015}$	$\underline{0.501 \pm 0.081}$
t-SNE	2	$\mathbf{0.862 \pm 0.011}$	$\underline{0.465 \pm 0.079}$	$\mathbf{0.832 \pm 0.019}$	$\mathbf{0.709 \pm 0.031}$	$\mathbf{0.895 \pm 0.007}$	$\mathbf{0.862 \pm 0.011}$	$\mathbf{0.503 \pm 0.064}$
Isomap	2	0.643 ± 0.020	0.115 ± 0.035	0.622 ± 0.023	0.249 ± 0.065	0.666 ± 0.018	0.643 ± 0.020	0.132 ± 0.041
Opt. App. Eigenmaps (Ours)	5	0.685 ± 0.022	0.226 ± 0.068	0.666 ± 0.026	0.348 ± 0.055	0.704 ± 0.020	0.685 ± 0.022	0.243 ± 0.073
Laplacian Eigenmaps	5	0.688 ± 0.024	0.160 ± 0.029	0.685 ± 0.026	0.390 ± 0.051	0.692 ± 0.025	0.688 ± 0.024	0.173 ± 0.029
PCA	5	0.677 ± 0.020	0.165 ± 0.044	0.653 ± 0.022	0.315 ± 0.064	0.704 ± 0.019	0.677 ± 0.020	0.186 ± 0.052
UMAP	5	$\mathbf{0.857 \pm 0.013}$	$\mathbf{0.501 \pm 0.089}$	$\mathbf{0.835 \pm 0.020}$	$\mathbf{0.704 \pm 0.040}$	$\underline{0.881 \pm 0.010}$	$\mathbf{0.857 \pm 0.013}$	$\mathbf{0.528 \pm 0.075}$
t-SNE	5	$\underline{0.853 \pm 0.011}$	$\underline{0.430 \pm 0.070}$	$\underline{0.821 \pm 0.019}$	$\underline{0.686 \pm 0.033}$	$\mathbf{0.887 \pm 0.007}$	$\underline{0.853 \pm 0.011}$	$\underline{0.469 \pm 0.056}$
Isomap	5	0.742 ± 0.017	0.291 ± 0.056	0.721 ± 0.021	0.463 ± 0.059	0.765 ± 0.015	0.742 ± 0.017	0.313 ± 0.056
Opt. App. Eigenmaps (Ours)	10	0.734 ± 0.018	0.309 ± 0.071	0.718 ± 0.022	0.456 ± 0.050	0.750 ± 0.015	0.734 ± 0.018	0.325 ± 0.073
Laplacian Eigenmaps	10	0.735 ± 0.020	0.223 ± 0.038	0.728 ± 0.020	0.475 ± 0.048	0.742 ± 0.025	0.735 ± 0.020	0.236 ± 0.038
PCA	10	0.724 ± 0.014	0.237 ± 0.051	0.698 ± 0.018	0.414 ± 0.062	0.752 ± 0.013	0.724 ± 0.014	0.262 ± 0.056
UMAP	10	$\mathbf{0.856 \pm 0.014}$	$\mathbf{0.494 \pm 0.088}$	$\mathbf{0.833 \pm 0.021}$	$\mathbf{0.702 \pm 0.040}$	$\mathbf{0.881 \pm 0.011}$	$\mathbf{0.856 \pm 0.014}$	$\mathbf{0.522 \pm 0.074}$
t-SNE	10	$\underline{0.844 \pm 0.012}$	$\underline{0.417 \pm 0.070}$	$\underline{0.812 \pm 0.019}$	$\underline{0.667 \pm 0.036}$	$\underline{0.879 \pm 0.008}$	$\underline{0.844 \pm 0.012}$	$\underline{0.456 \pm 0.057}$
Isomap	10	0.788 ± 0.013	0.381 ± 0.062	0.767 ± 0.018	0.560 ± 0.054	0.810 ± 0.012	0.788 ± 0.013	0.403 ± 0.059

Table 12. Average manifold learning results on CIFAR100 with DINOv2 embeddings. The mean and standard deviation over 32 runs is provided. The best and second best results are in bold and underlined respectively.

CIFAR100 - DINOv2								
Method	k	NMI	ARI	Comp.	AMI	Homo.	V-Meas.	FMI
Opt. App. Eigenmaps (Ours)	2	0.568 ± 0.016	0.164 ± 0.036	0.576 ± 0.016	0.412 ± 0.046	0.561 ± 0.020	0.568 ± 0.016	0.191 ± 0.038
Laplacian Eigenmaps	2	0.590 ± 0.032	0.186 ± 0.037	0.577 ± 0.032	0.426 ± 0.050	0.603 ± 0.034	0.590 ± 0.032	0.206 ± 0.038
PCA	2	0.492 ± 0.017	0.098 ± 0.019	0.472 ± 0.020	0.273 ± 0.046	0.515 ± 0.015	0.492 ± 0.017	0.118 ± 0.024
UMAP	2	$\mathbf{0.745 \pm 0.016}$	$\mathbf{0.433 \pm 0.040}$	$\mathbf{0.716 \pm 0.018}$	$\mathbf{0.635 \pm 0.040}$	$\mathbf{0.776 \pm 0.021}$	$\mathbf{0.745 \pm 0.016}$	$\mathbf{0.457 \pm 0.043}$
t-SNE	2	$\underline{0.740 \pm 0.014}$	$\underline{0.412 \pm 0.038}$	$\underline{0.708 \pm 0.016}$	$\underline{0.626 \pm 0.038}$	$\underline{0.775 \pm 0.018}$	$\underline{0.740 \pm 0.014}$	$\underline{0.439 \pm 0.041}$
Isomap	2	0.533 ± 0.018	0.142 ± 0.032	0.512 ± 0.020	0.332 ± 0.051	0.556 ± 0.018	0.533 ± 0.018	0.163 ± 0.036
Opt. App. Eigenmaps (Ours)	5	0.583 ± 0.017	0.176 ± 0.029	0.602 ± 0.023	0.442 ± 0.034	0.565 ± 0.016	0.583 ± 0.017	0.208 ± 0.030
Laplacian Eigenmaps	5	0.640 ± 0.019	0.239 ± 0.036	0.622 ± 0.018	0.492 ± 0.047	0.659 ± 0.024	0.640 ± 0.019	0.258 ± 0.039
PCA	5	0.601 ± 0.013	0.206 ± 0.031	0.576 ± 0.015	0.428 ± 0.049	0.629 ± 0.015	0.601 ± 0.013	0.228 ± 0.036
UMAP	5	$\mathbf{0.750 \pm 0.016}$	$\mathbf{0.447 \pm 0.041}$	$\mathbf{0.722 \pm 0.018}$	$\mathbf{0.644 \pm 0.039}$	$\mathbf{0.781 \pm 0.020}$	$\mathbf{0.750 \pm 0.016}$	$\mathbf{0.470 \pm 0.043}$
t-SNE	5	$\underline{0.736 \pm 0.015}$	$\underline{0.403 \pm 0.036}$	$\underline{0.705 \pm 0.017}$	$\underline{0.621 \pm 0.037}$	$\underline{0.771 \pm 0.018}$	$\underline{0.736 \pm 0.015}$	$\underline{0.430 \pm 0.039}$
Isomap	5	0.647 ± 0.017	0.291 ± 0.048	0.622 ± 0.018	0.495 ± 0.049	0.673 ± 0.020	0.647 ± 0.017	0.313 ± 0.052
Opt. App. Eigenmaps (Ours)	10	0.613 ± 0.013	0.214 ± 0.039	0.622 ± 0.016	0.477 ± 0.036	0.605 ± 0.017	0.613 ± 0.013	0.238 ± 0.040
Laplacian Eigenmaps	10	0.669 ± 0.018	0.273 ± 0.038	0.649 ± 0.017	0.531 ± 0.048	0.690 ± 0.024	0.669 ± 0.018	0.292 ± 0.041
PCA	10	0.649 ± 0.018	0.268 ± 0.042	0.622 ± 0.017	0.495 ± 0.054	0.679 ± 0.023	0.649 ± 0.018	0.292 ± 0.047
UMAP	10	$\mathbf{0.751 \pm 0.015}$	$\mathbf{0.448 \pm 0.038}$	$\mathbf{0.722 \pm 0.017}$	$\mathbf{0.644 \pm 0.036}$	$\mathbf{0.781 \pm 0.018}$	$\mathbf{0.751 \pm 0.015}$	$\mathbf{0.472 \pm 0.040}$
t-SNE	10	$\underline{0.734 \pm 0.019}$	$\underline{0.400 \pm 0.044}$	$\underline{0.703 \pm 0.020}$	$\underline{0.617 \pm 0.044}$	$\underline{0.768 \pm 0.023}$	$\underline{0.734 \pm 0.019}$	$\underline{0.426 \pm 0.047}$
Isomap	10	0.689 ± 0.018	0.355 ± 0.055	0.664 ± 0.017	0.556 ± 0.050	0.717 ± 0.023	0.689 ± 0.018	0.376 ± 0.059

Table 13. Average manifold learning results on CIFAR100 with CLIP embeddings. The mean and standard deviation over 32 runs is provided. The best and second best results are in bold and underlined respectively.

CIFAR100 - CLIP								
Method	k	NMI	ARI	Comp.	AMI	Homo.	V-Meas.	FMI
Opt. App. Eigenmaps (Ours)	2	0.499 ± 0.021	0.111 ± 0.024	0.488 ± 0.024	0.300 ± 0.040	0.511 ± 0.020	0.499 ± 0.021	0.132 ± 0.027
Laplacian Eigenmaps	2	0.392 ± 0.064	0.055 ± 0.028	0.414 ± 0.068	0.208 ± 0.080	0.373 ± 0.063	0.392 ± 0.064	0.088 ± 0.035
PCA	2	0.423 ± 0.028	0.052 ± 0.012	0.406 ± 0.031	0.177 ± 0.036	0.442 ± 0.025	0.423 ± 0.028	0.072 ± 0.016
UMAP	2	$\underline{0.656 \pm 0.022}$	$\mathbf{0.320 \pm 0.045}$	$\underline{0.628 \pm 0.024}$	$\underline{0.506 \pm 0.049}$	$\underline{0.686 \pm 0.023}$	$\underline{0.656 \pm 0.022}$	$\underline{0.345 \pm 0.048}$
t-SNE	2	$\mathbf{0.668 \pm 0.021}$	$\underline{0.320 \pm 0.038}$	$\mathbf{0.639 \pm 0.024}$	$\mathbf{0.523 \pm 0.044}$	$\mathbf{0.701 \pm 0.021}$	$\mathbf{0.668 \pm 0.021}$	$\mathbf{0.346 \pm 0.041}$
Isomap	2	0.464 ± 0.019	0.083 ± 0.019	0.448 ± 0.022	0.240 ± 0.044	0.481 ± 0.017	0.464 ± 0.019	0.103 ± 0.024
Opt. App. Eigenmaps (Ours)	5	0.555 ± 0.019	0.159 ± 0.027	0.552 ± 0.022	0.388 ± 0.040	0.557 ± 0.020	0.555 ± 0.019	0.180 ± 0.030
Laplacian Eigenmaps	5	0.487 ± 0.051	0.101 ± 0.031	0.520 ± 0.047	0.333 ± 0.059	0.459 ± 0.056	0.487 ± 0.051	0.140 ± 0.032
PCA	5	0.504 ± 0.020	0.116 ± 0.026	0.481 ± 0.023	0.286 ± 0.048	0.528 ± 0.019	0.504 ± 0.020	0.138 ± 0.030
UMAP	5	$\mathbf{0.667 \pm 0.021}$	$\mathbf{0.342 \pm 0.041}$	$\mathbf{0.641 \pm 0.023}$	$\mathbf{0.525 \pm 0.044}$	$\underline{0.696 \pm 0.021}$	$\mathbf{0.667 \pm 0.021}$	$\mathbf{0.365 \pm 0.045}$
t-SNE	5	$\underline{0.665 \pm 0.021}$	$\underline{0.310 \pm 0.039}$	$\underline{0.636 \pm 0.023}$	$\underline{0.519 \pm 0.044}$	$\mathbf{0.698 \pm 0.021}$	$\underline{0.665 \pm 0.021}$	$\underline{0.336 \pm 0.042}$
Isomap	5	0.561 ± 0.019	0.176 ± 0.036	0.541 ± 0.021	0.375 ± 0.054	0.582 ± 0.020	0.561 ± 0.019	0.197 ± 0.041
Opt. App. Eigenmaps (Ours)	10	0.566 ± 0.021	0.173 ± 0.037	0.577 ± 0.023	0.415 ± 0.042	0.555 ± 0.023	0.566 ± 0.021	0.200 ± 0.037
Laplacian Eigenmaps	10	0.551 ± 0.031	0.137 ± 0.037	0.579 ± 0.025	0.407 ± 0.049	0.526 ± 0.039	0.551 ± 0.031	0.174 ± 0.036
PCA	10	0.557 ± 0.018	0.172 ± 0.035	0.532 ± 0.021	0.363 ± 0.047	0.584 ± 0.017	0.557 ± 0.018	0.195 ± 0.039
UMAP	10	$\mathbf{0.667 \pm 0.021}$	$\mathbf{0.343 \pm 0.045}$	$\mathbf{0.641 \pm 0.024}$	$\mathbf{0.525 \pm 0.044}$	$\mathbf{0.696 \pm 0.021}$	$\mathbf{0.667 \pm 0.021}$	$\mathbf{0.366 \pm 0.048}$
t-SNE	10	$\underline{0.653 \pm 0.027}$	$\underline{0.292 \pm 0.049}$	$\underline{0.625 \pm 0.028}$	$\underline{0.501 \pm 0.056}$	$\underline{0.685 \pm 0.028}$	$\underline{0.653 \pm 0.027}$	$\underline{0.318 \pm 0.053}$
Isomap	10	0.588 ± 0.019	0.211 ± 0.039	0.567 ± 0.021	0.414 ± 0.051	0.611 ± 0.021	0.588 ± 0.019	0.232 ± 0.044

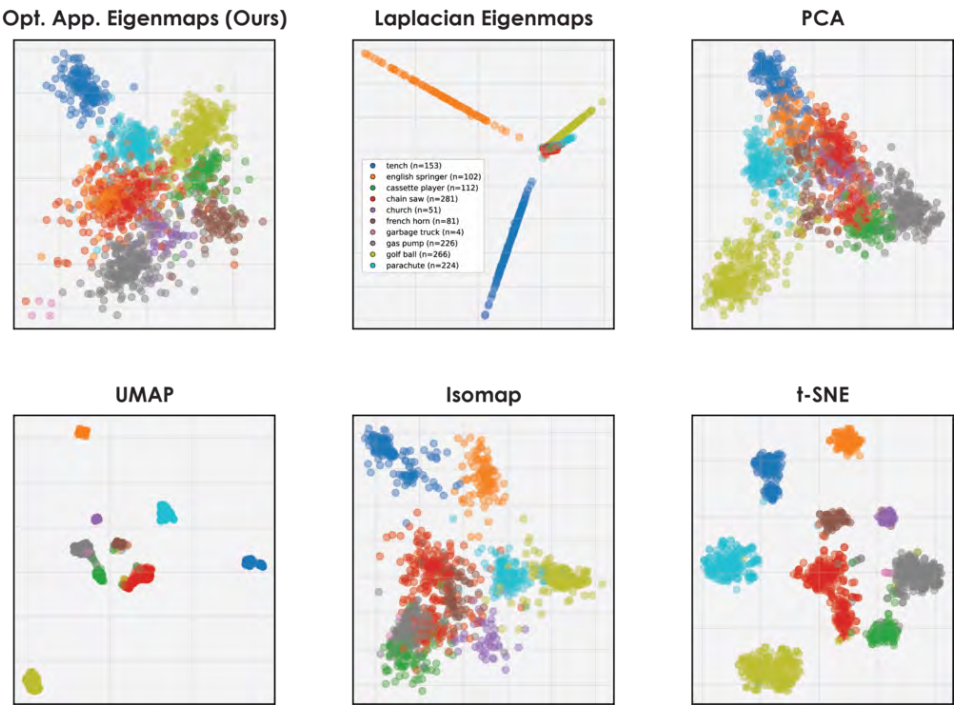


Figure 23. 2D visualization of a random subset of the Imagenette dataset using CLIP embeddings. Points are colored by ground truth class labels. Our learned spectral basis (Optimal Approximation Eigenmaps) produces meaningful clusters that separate the image manifold according to semantic content, with less overlap than in PCA and Isomap, while accurately preserving the smooth manifold structure without artificially overclustering it as in t-SNE, Umap, and Laplacian Eigenmaps.

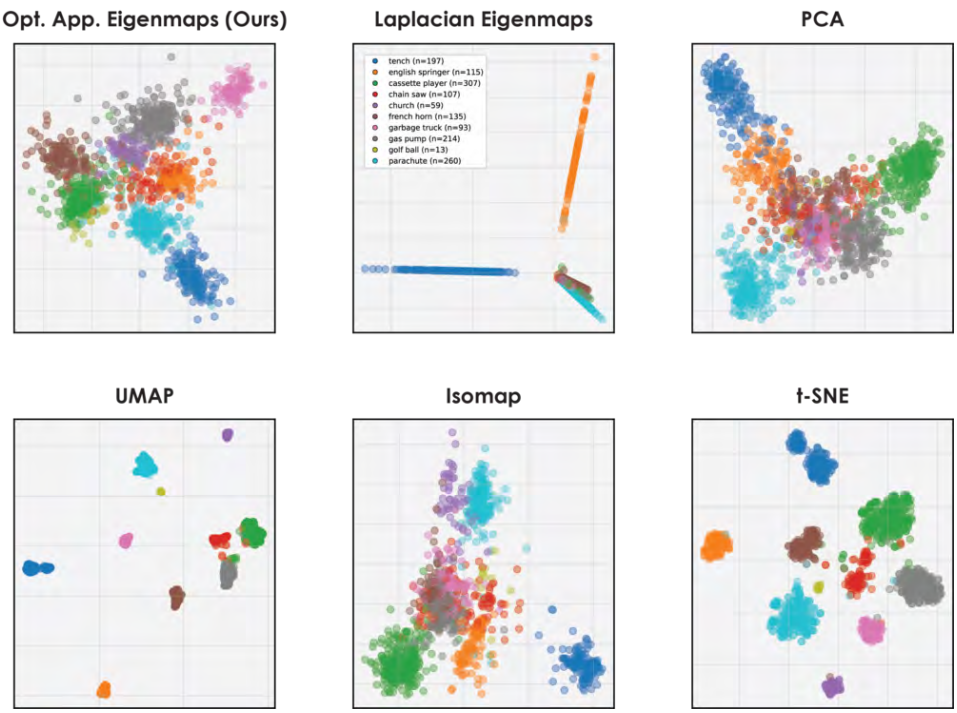


Figure 24. 2D visualization of another random subset of the Imagenette dataset using CLIP embeddings.

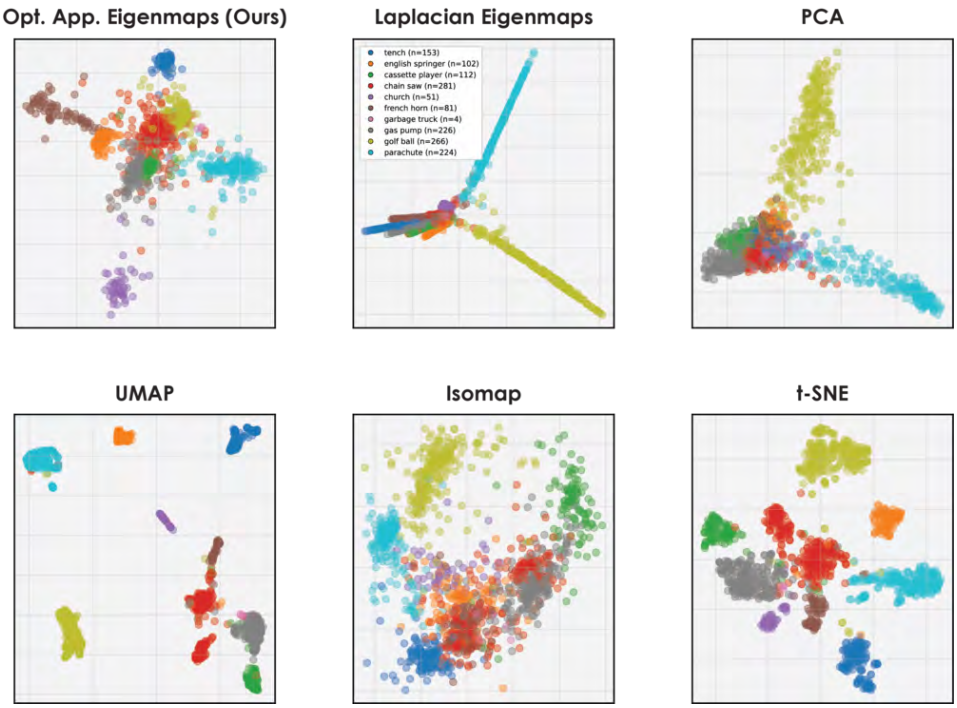


Figure 25. 2D visualization of a random subset of the Imagenette dataset using DINOv2 embeddings.

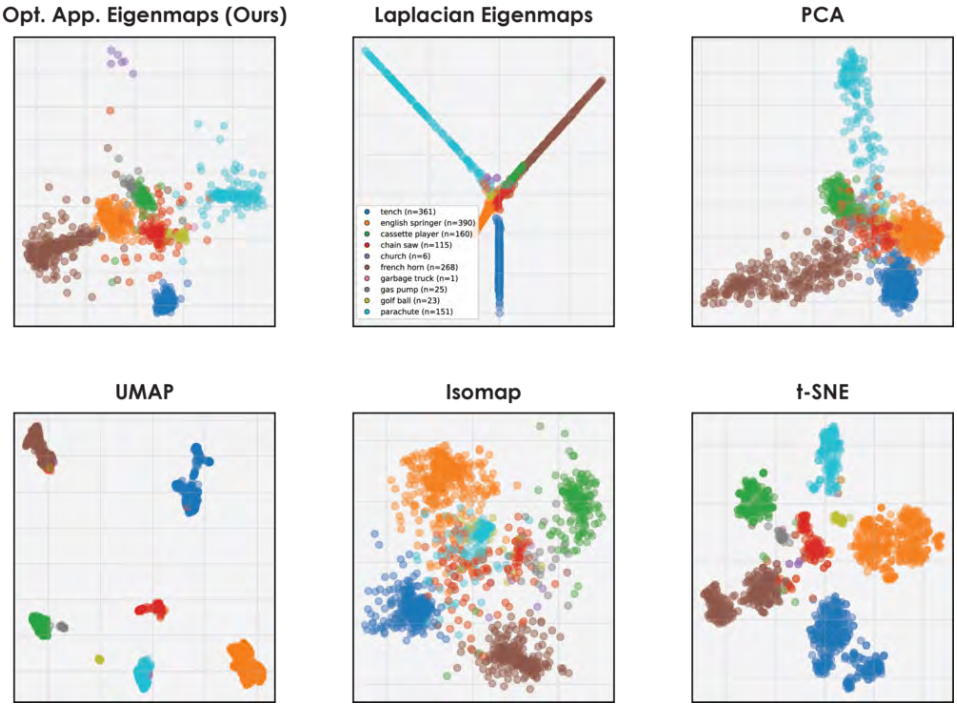


Figure 26. 2D visualization of another random subset of the Imagenette dataset using DINOv2 embeddings.

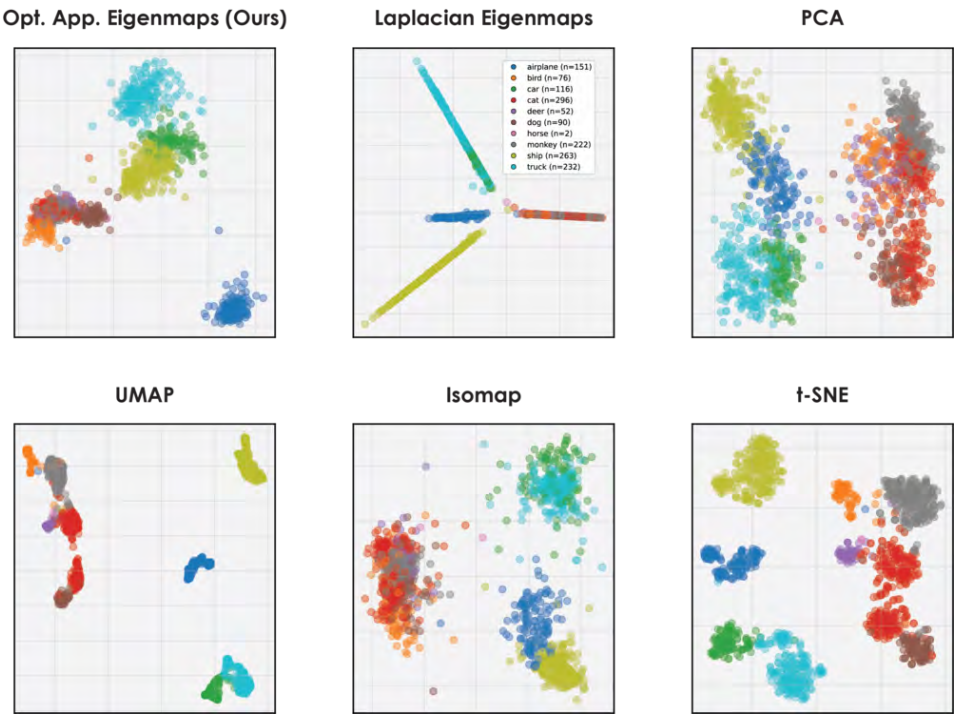


Figure 27. 2D visualization of a random subset of the STL-10 dataset using CLIP embeddings.

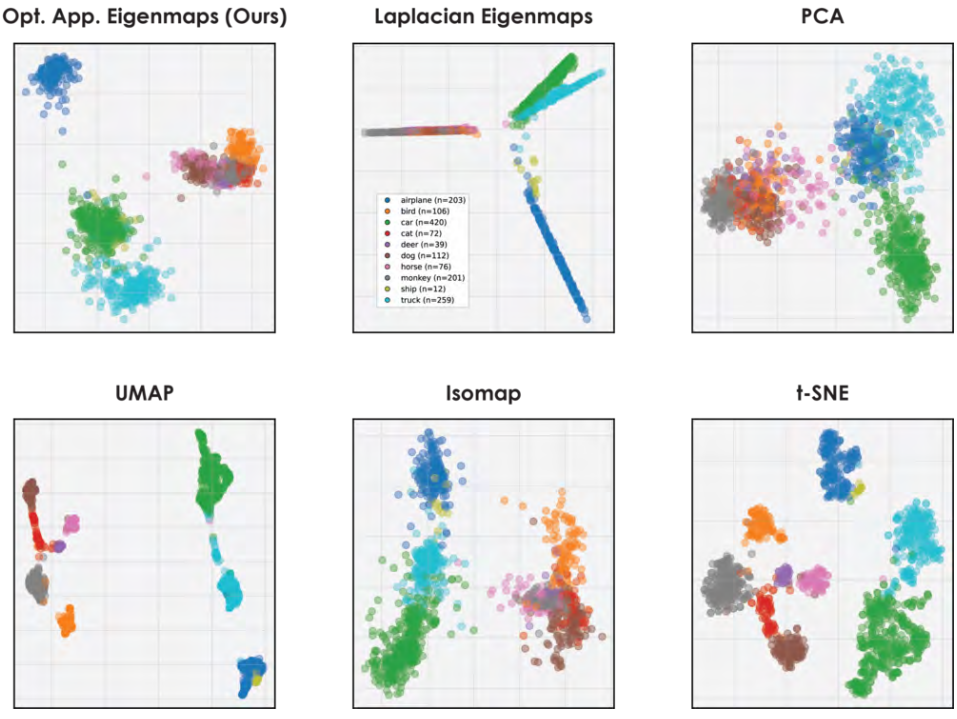


Figure 28. 2D visualization of another random subset of the STL-10 dataset using CLIP embeddings.

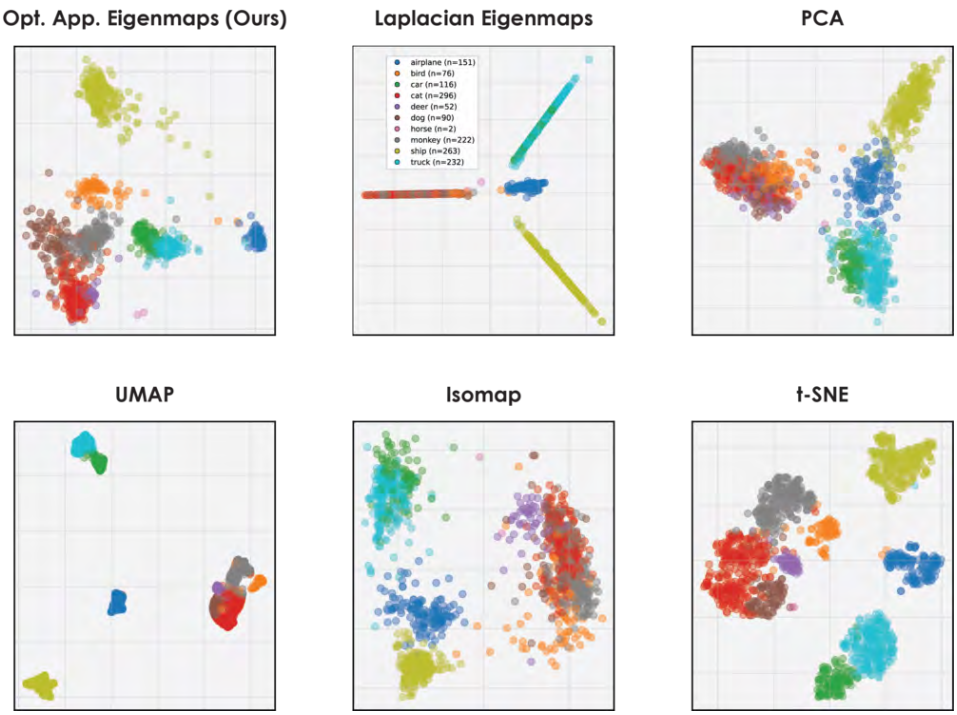


Figure 29. 2D visualization of a random subset of the STL-10 dataset using DINOv2 embeddings.

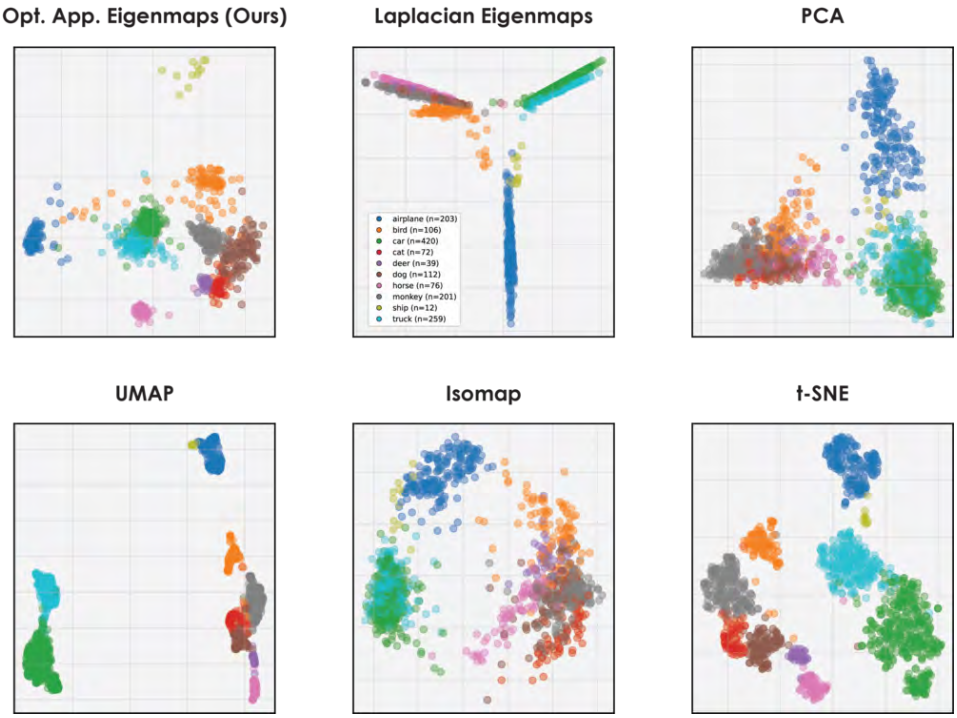


Figure 30. 2D visualization of another random subset of the STL-10 dataset using DINOv2 embeddings.